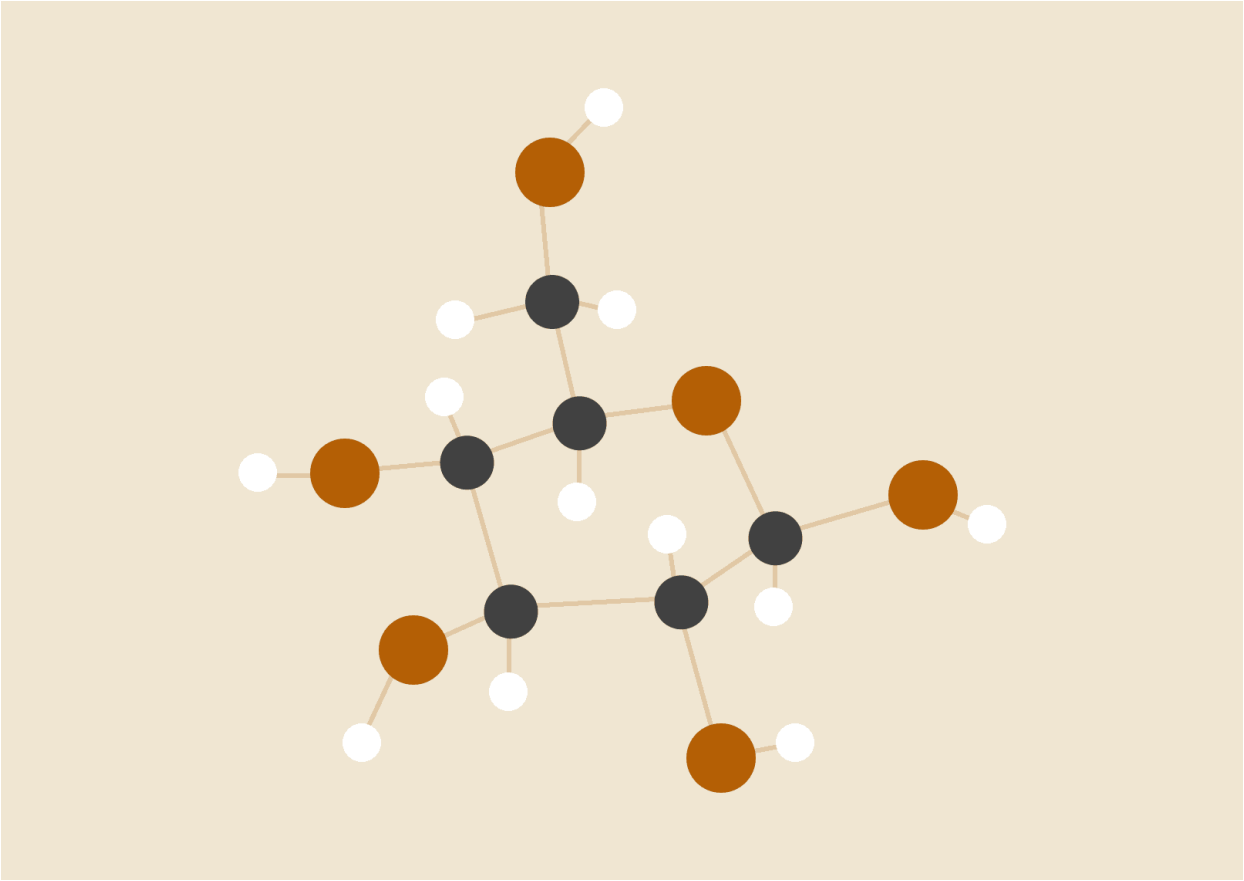
PROJECT REPORT



Abdulrahman Emad [2023/-]

Janat Yasser [2023/07306]

Aya Shehab [2023/07-]

Abdulah Wael [2023/]

Youssef Kareem [2023/0-]

16.19.2025

Course: Artificial Intelligence

Dataset: Hotel Bookings Dataset

Objective: Predict whether a hotel booking will be canceled using machine learning models

INTRODUCTION

This document explores the topic of hotel cancellation prediction and how it was designed as a project, algorithm used, models etc.

Hotel cancellations can cause heavy losses and challenges for these hotels; by building a software that analyzes patterns in behavior, characteristics and trends; these cancellations can be identified to pursue more efficient hotel service by enabling retention services.

Our data set contains 119,390 bookings with 33 columns and key variables.

Dataset Overview

Initial Dataset Characteristics

Data > Preprocessing > EDA > Feature Engineering > Model Building > Evaluation

When designing the overview we kept a few things in mind, this project followed a structured pipeline where data cleaning came in first to clean and polish our data for maximum efficiency, it did this by handling missing values and outliers.

Then comes exploratory analysis which identifies patterns and correlations; feature engineering then uses the information to create predictive features and genetic algorithms to select the optimal feature subsets to maximize model accuracy and productivity.

Then we use multiple classification models that will be discussed below, all of which are trained and evaluated to give us the most accurate data.

We designed this architecture with 2 goals in mind, robust healthy predictions and preventions of data leakage and/or over fitting.

The dataset was loaded from an Excel file containing hotel booking information. After initial inspection, we found:

- **Total Records:** 119,390 bookings
- **Total Features:** 33 columns
- **Target Variable:** `is_canceled` (0 = Not Canceled, 1 = Canceled)

The dataset includes various types of information:

- Booking details (lead time, arrival dates, length of stay)
- Customer information (adults, children, country of origin)
- Hotel-specific data (hotel type, city, room types)
- Booking channel information (agent, company, distribution channel)
- Financial data (average daily rate, deposit type)

Missing Values Analysis

During the initial inspection, we identified missing values in several columns:

Column	Missing Count	Missing Percentage
company	112,593	94.31%
agent	16,340	13.69%
country	488	0.41%
children	4	0.003%

Decision Made: We decided to drop the `company` and `agent` columns entirely due to their extremely high missing rates. For `country` and `children`, we applied imputation techniques since the missing percentage was manageable.

Data Preprocessing

Handling Missing Values

We adopted a strategic approach to handle missing values based on the nature of each column:

For Numerical Columns:

- Used median imputation to fill missing values
- This approach is robust against outliers and maintains the central tendency of the data

For Categorical Columns:

- Used mode (most frequent value) imputation.
- This ensures the most common category fills the gaps

```
            # Numerical columns - median imputation
for i in df.select_dtypes(include="number").columns: df[i] =
                df[i].fillna(df[i].median())
            # Categorical columns - mode imputation
for i in df.select_dtypes(include="object").columns: df[i] =
                df[i].fillna(df[i].mode()[0])
```

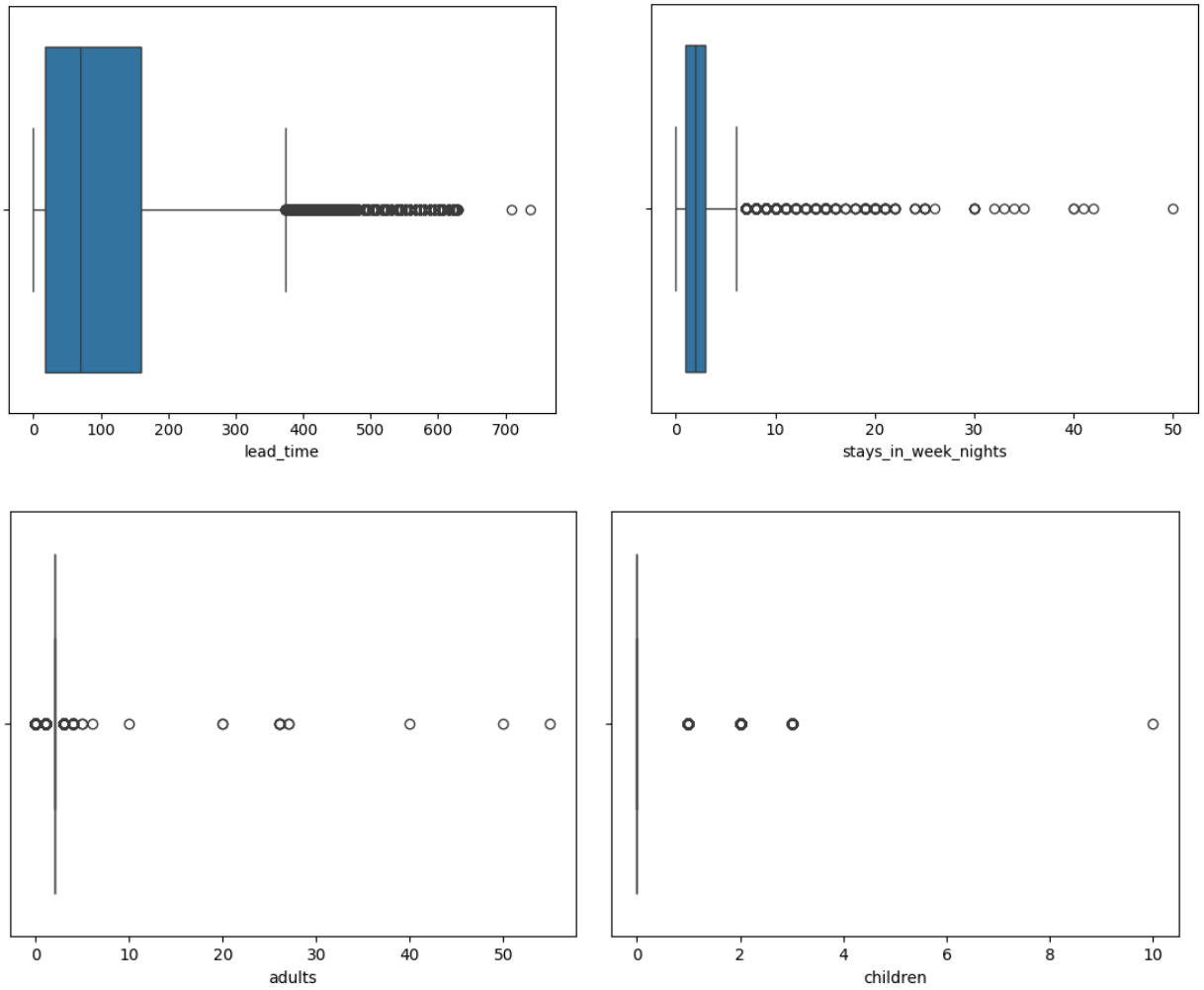
After this step, the dataset had zero missing values, making it ready for further processing.

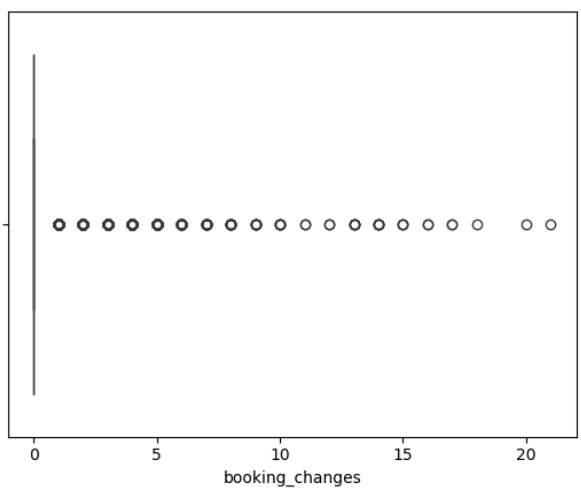
Duplicate Removal

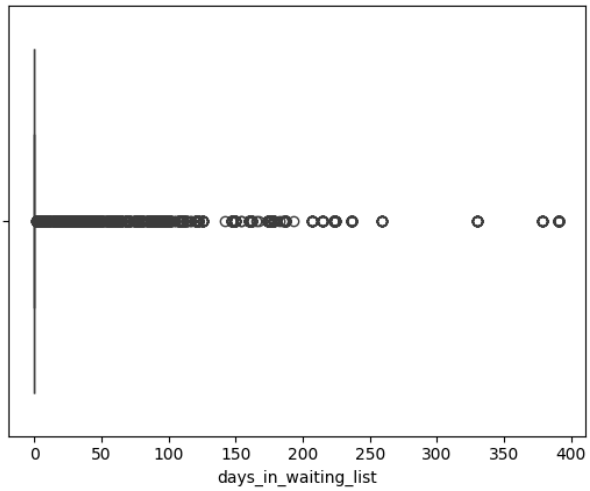
We checked for duplicate records in the dataset and found 63 duplicate rows. These were removed to ensure data quality and prevent bias in our models. Keeping duplicates could artificially inflate certain patterns and lead to overfitting.

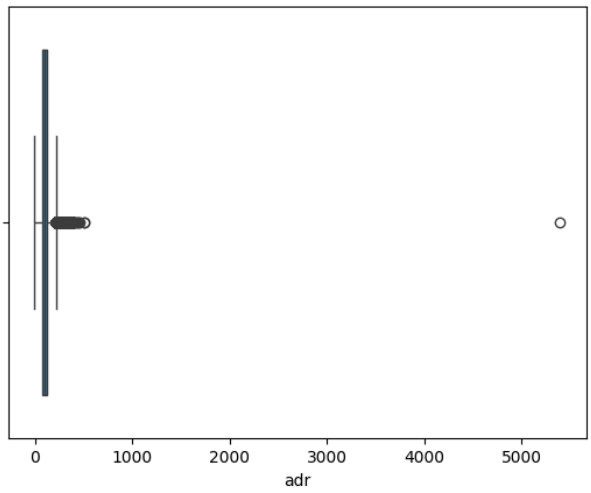
Outlier Detection and Treatment

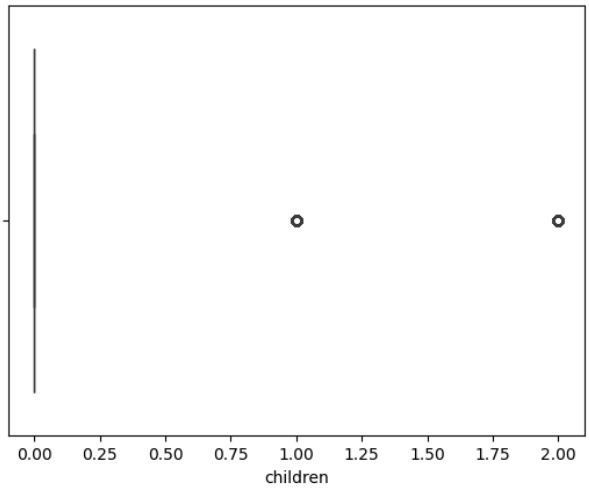
Outliers can significantly affect model performance, especially for distance-based algorithms like KNN. We identified outliers using boxplots for all numerical features.




0
5
10
15
20
booking_changes


0
50
100
150
200
250
300
350
400
days_in_waiting_list

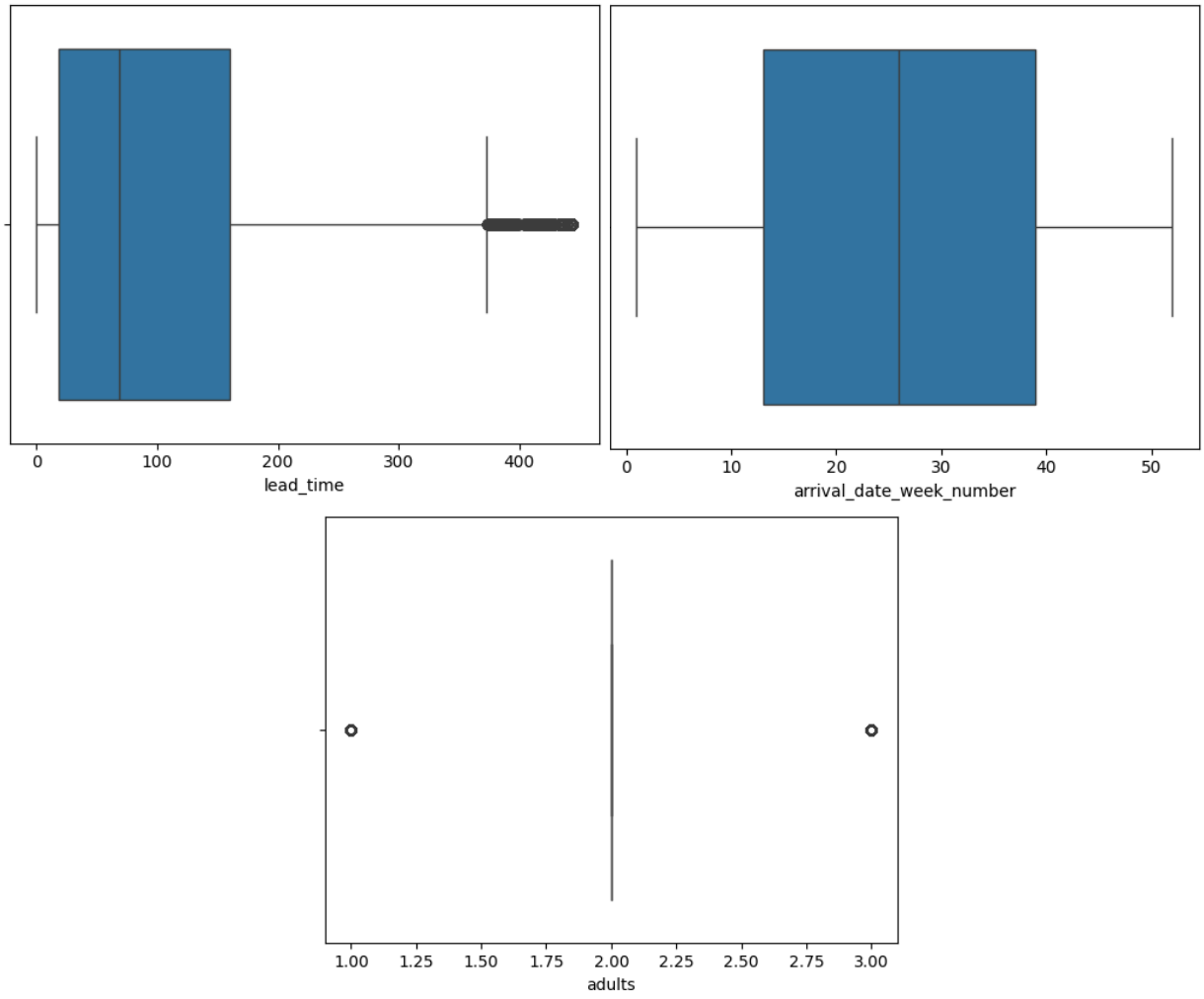

0
1000
2000
3000
4000
5000
adr

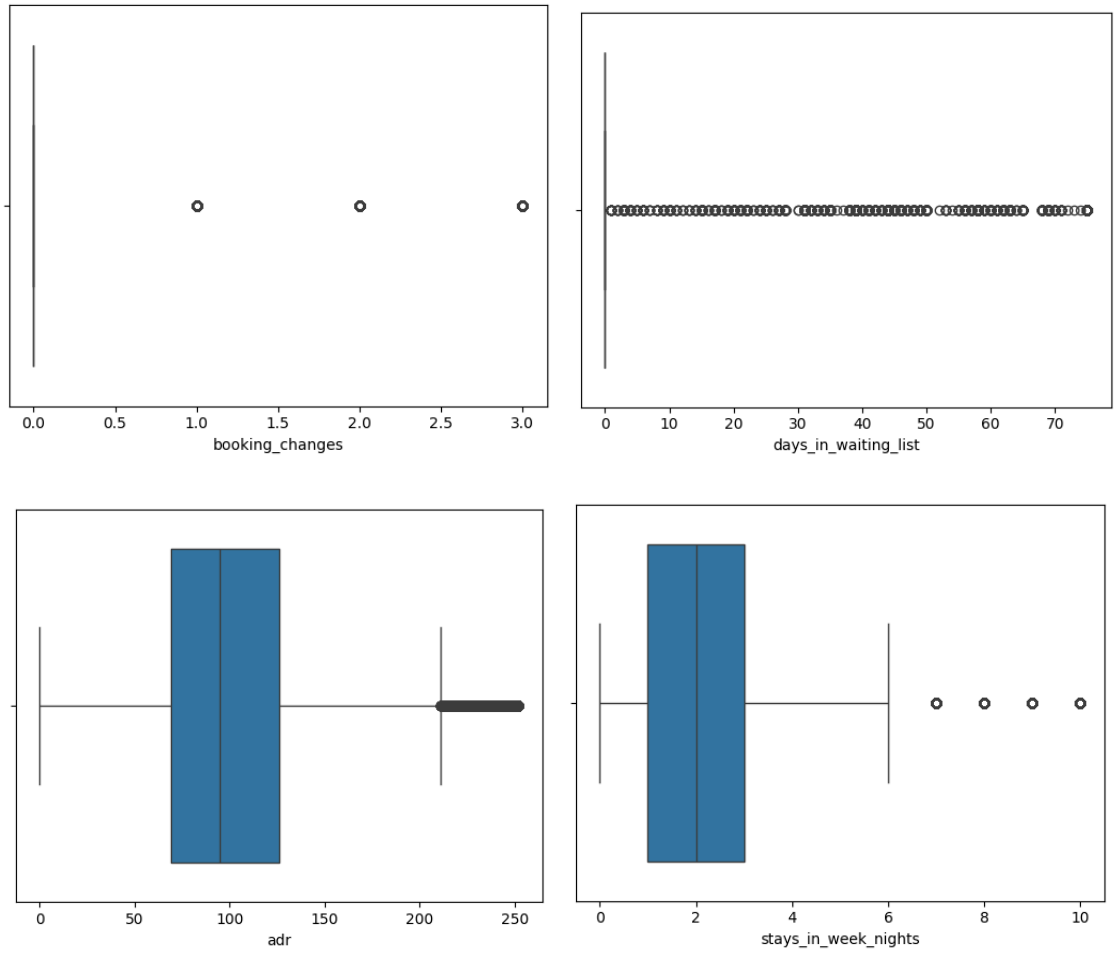

0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
children

Treatment Method: Instead of removing outliers entirely (which could lose valuable information), we used the clipping method at the 1st and 99th percentiles. This approach caps extreme values while preserving the overall distribution.

```
for i in need_to_remove_outliers: max_value = df[i].quantile(0.99)
min_value = df[i].quantile(0.01) df[i] = df[i].clip(upper=max_value,
                                lower=min_value)
```

After treatment, the outliers were significantly reduced, resulting in more stable distributions suitable for modeling.




0.0
0.5
1.0
1.5
2.0
2.5
3.0
booking_changes
0
10
20
30
40
50
60
70
days_in_waiting_list
0
50
100
150
200
250
adr
0
2
4
6
8
10
stays_in_week_nights

Feature Engineering

We created several new features to enhance model performance by capturing additional patterns in the data:

1. Hotel Frequency Encoding

```
freq = df['hotel'].value_counts() df['hotel_freq'] =
                df['hotel'].map(freq)
```

This feature captures how popular each hotel is based on booking frequency.

2. Month Number Conversion

```
month_map = {"January": 1, "February": 2, ..., "December": 12}
              df["arrival_date_month_number"] =
          df["arrival_date_month"].map(month_map)
```

Converted month names to numbers for better numerical processing.

3. Room Assignment Check

```
df["got_reserved_room"] = (df["reserved_room_type"] ==
df["assigned_room_type"]) df["got_reserved_room_num"] =
   df["got_reserved_room"].map({False: 0, True: 1})
```

This binary feature indicates whether customers received their requested room type.

4. Country Frequency

```
country_freq = df['country'].value_counts()
df['country_freq'] = df['country'].map(country_freq)
```

Encoded countries by their booking frequency rather than one-hot encoding due to high cardinality.

5. Additional Composite Features

`total_stay`: Combined weekend and weeknight stays

`total_guests`: Sum of adults, children, and babies

`is_family`: Binary indicator if booking includes children or babies

`adr_level`: Categorized average daily rate into three levels (low, medium, high)

Encoding Categorical Variables

Different encoding strategies were applied based on the nature and cardinality of categorical features:

Label Encoding (for ordinal or low-cardinality features):

- meal
- market_segment
- deposit_type
- customer_type
- distribution_channel

One-Hot Encoding (for nominal features with moderate cardinality):

- city (15 unique cities)

Frequency Encoding (for high-cardinality features):

- hotel
- country

After encoding, all categorical variables were converted to numerical format, and the original categorical columns were dropped. The final dataset contained only numerical values suitable for machine learning algorithms.

Data Cleaning Summary

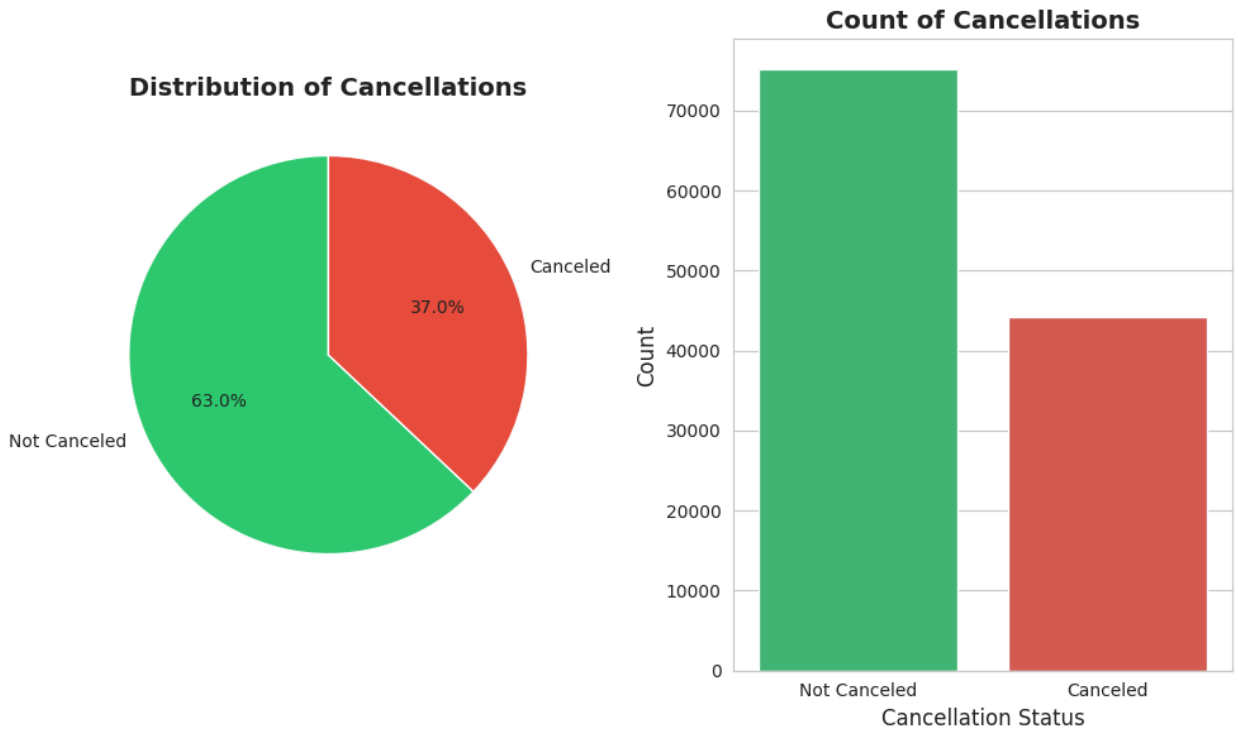
After preprocessing, our cleaned dataset had:

- Zero missing values
- No duplicate records
- **Treated outliers** in key numerical features
- **All numerical features** ready for modeling
- Enhanced with engineered features

Exploratory Data Analysis (EDA)

Target Variable Distribution

The first and most important analysis was understanding our target variable distribution.



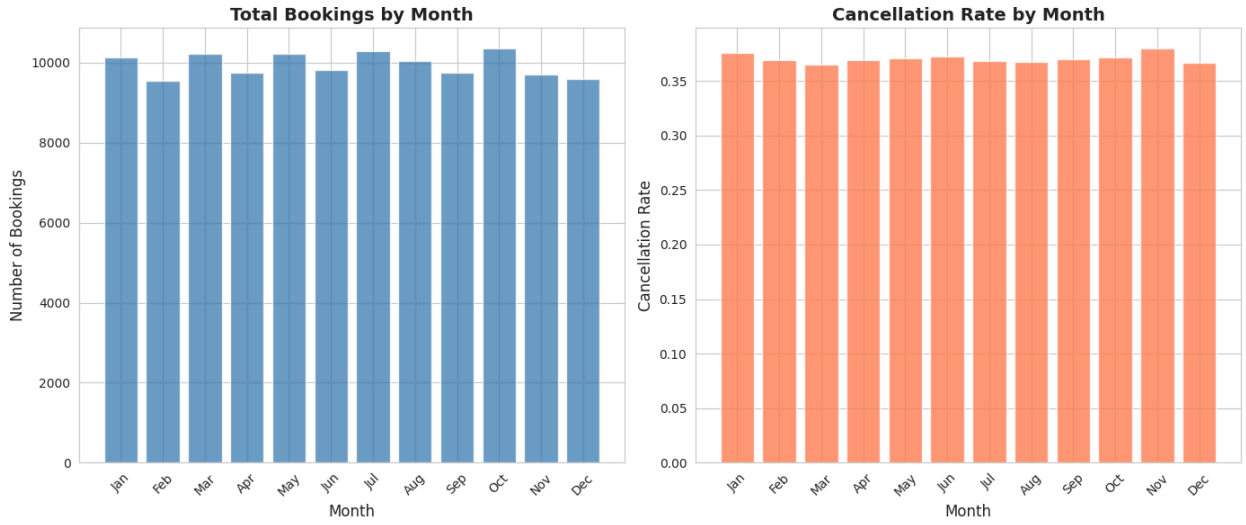
Key Findings:

- **Total Bookings:** 119,390
- **Canceled Bookings:** 44,224 (37.04%)
- **Not Canceled Bookings:** 75,166 (62.96%)

The dataset shows a moderate class imbalance with approximately 37% cancellations. This is a realistic representation of hotel booking behavior. While not severely imbalanced, we need to be aware of this when building our models and may need to apply techniques like SMOTE if performance issues arise.

Temporal Patterns

Monthly Booking Trends



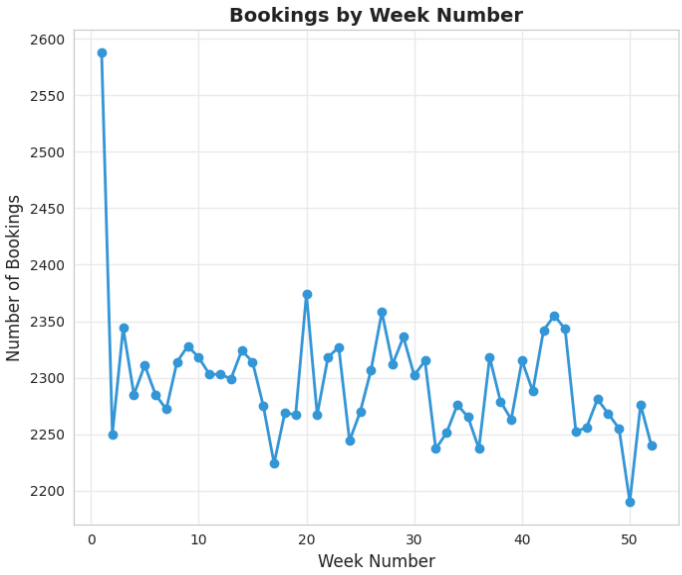
We analyzed how bookings and cancellations vary across different months of the year.

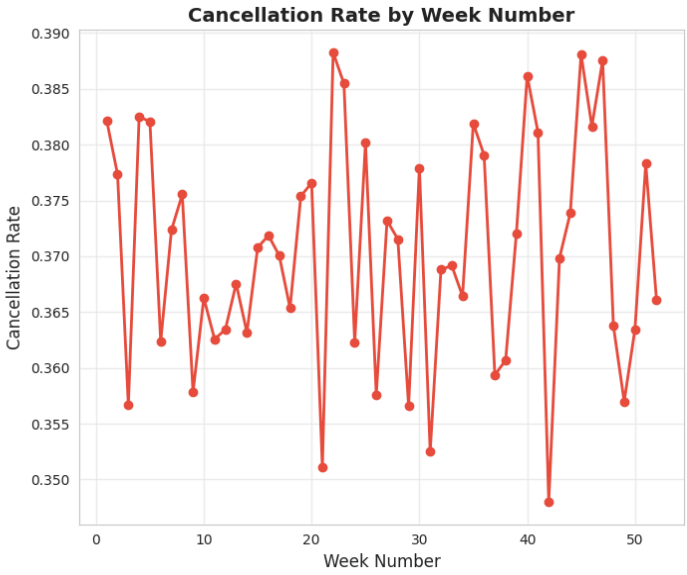
Observations:

- **Peak Booking Months:** October, July, and March show the highest booking volumes
- **Highest Cancellation Rates:** November, January, and June experience the most cancellations
- **Insight:** Summer and holiday months attract more bookings but also face higher cancellation rates, possibly due to flexible travel plans during vacation periods

This temporal pattern suggests that seasonality plays a role in both booking behavior and cancellation likelihood. Models might consider the arrival month as an important feature.

Weekly Patterns



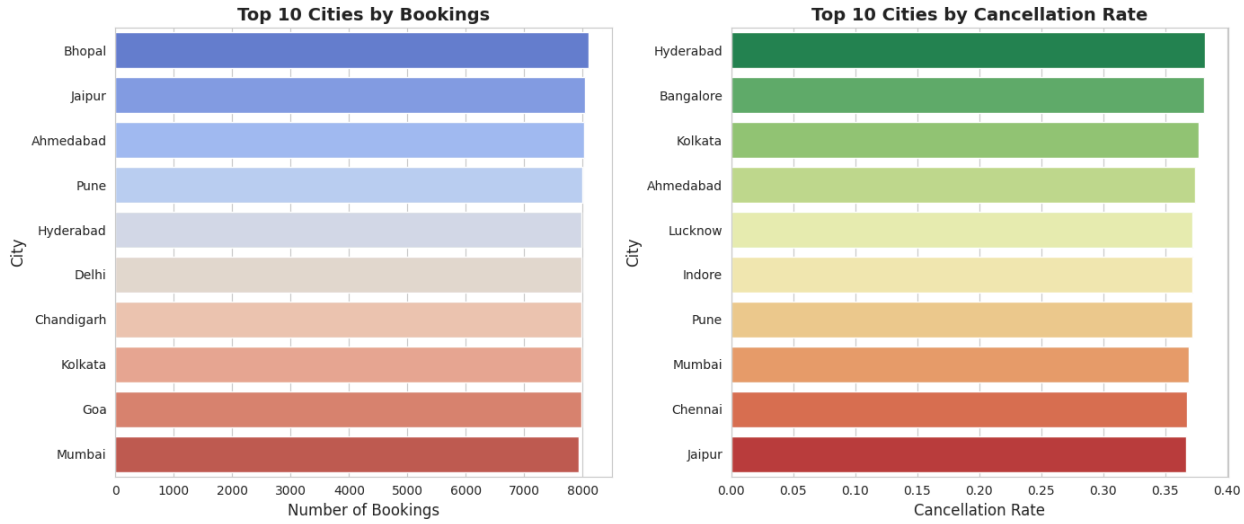


Weekly analysis revealed:

- Booking volumes fluctuate throughout the year with distinct peaks
- Cancellation rates also vary by week, suggesting certain periods are more uncertain
- Holiday weeks and summer periods show both high bookings and high cancellations

Geographic Analysis

City-wise Distribution



Most Popular Cities:

1. Bhopal
2. Jaipur
3. Ahmedabad

These cities attract the most bookings, indicating they are major tourist or business destinations.

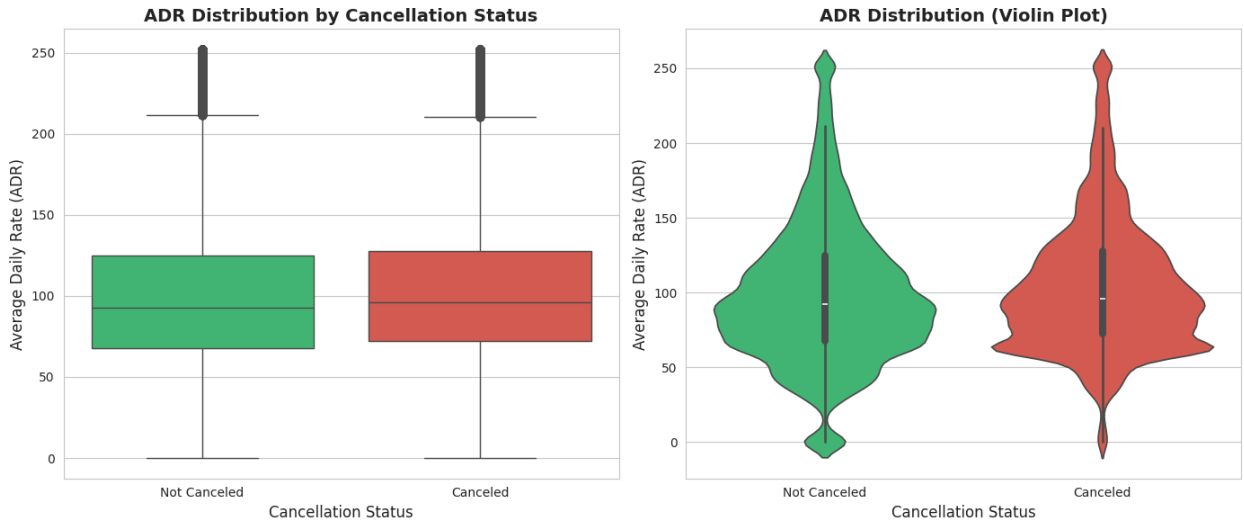
Cities with Highest Cancellation Rates:

1. Hyderabad
2. Bangalore
3. Kolkata

Cities with the most bookings don't necessarily have the highest cancellation rates. Bangalore and Hyderabad, being tech hubs, might see more business travel cancellations due to changing work plans.

Pricing Analysis

Average Daily Rate (ADR) Impact

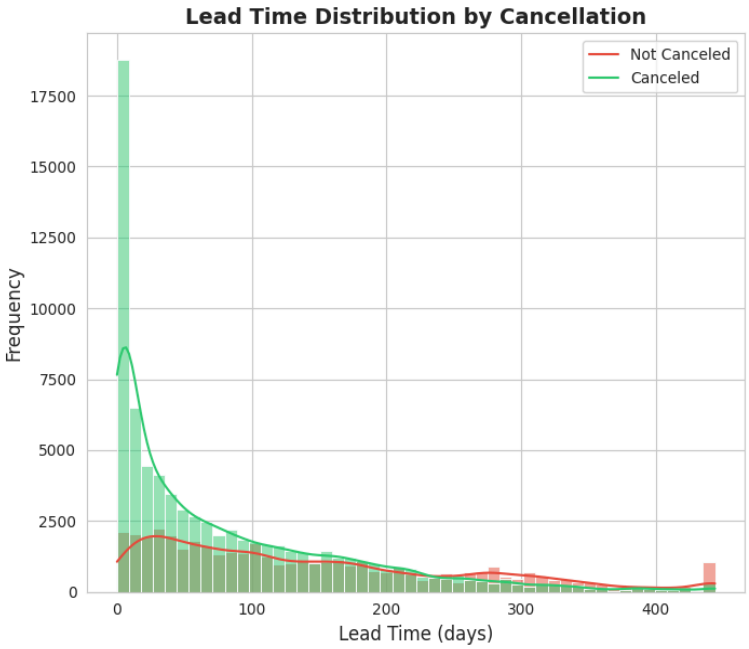


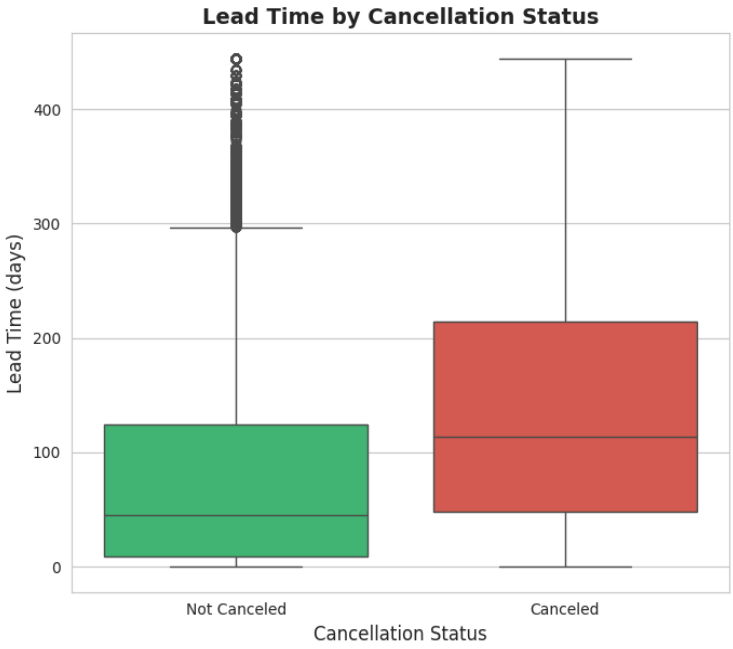
Statistical Findings:

- **Mean ADR for Not Canceled:** $99.65
- **Mean ADR for Canceled:** $104.54

Concluded: Canceled bookings tend to have HIGHER average daily rates. This is a significant finding that suggests price sensitivity plays a role in cancellation decisions. When customers book expensive rooms, they might be more likely to cancel if they find better deals or if their plans change. This contradicts the assumption that expensive bookings are more committed.

Lead Time Analysis





Lead time (days between booking and arrival) showed a very strong relationship with cancellations:

Statistical Findings:

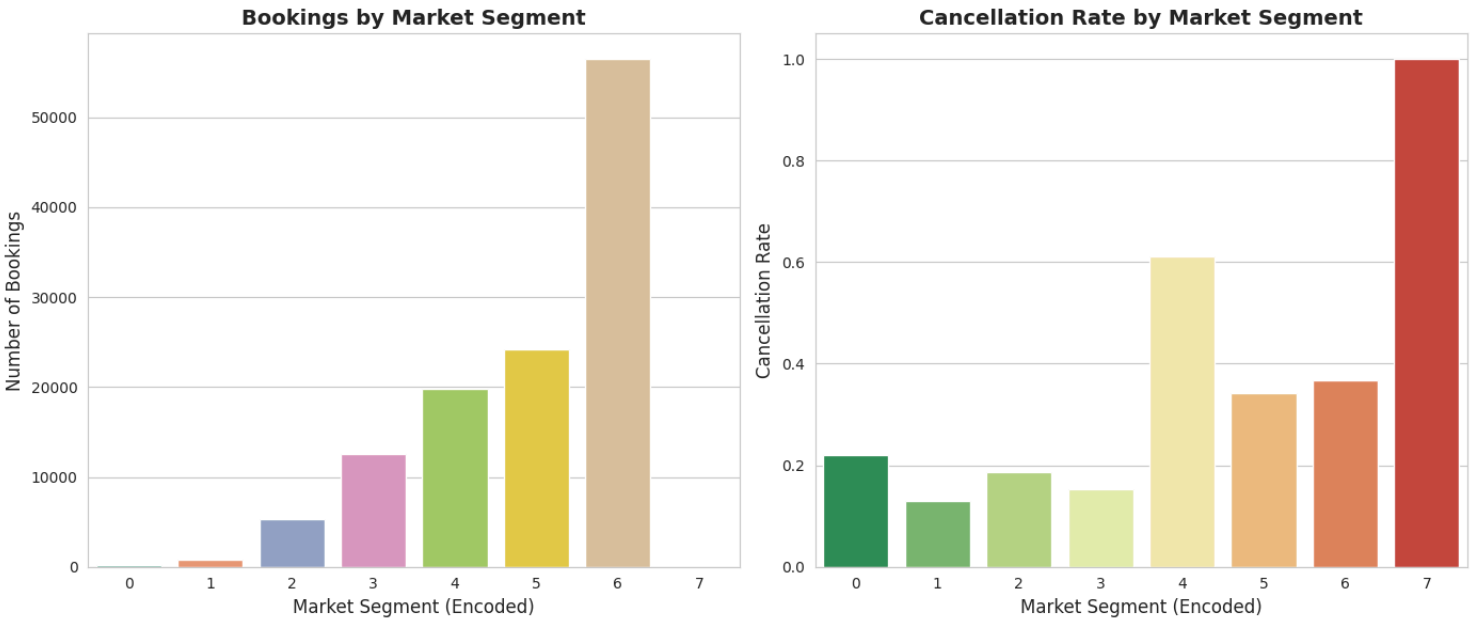
- **Mean Lead Time for Not Canceled:** 79.86 days
- **Mean Lead Time for Canceled:** 143.44 days

Concluded: Bookings made far in advance (longer lead times) have significantly higher cancellation rates. This makes intuitive sense - when people book months ahead, circumstances are more likely to change. Plans made closer to the arrival date are more concrete and committed.

This is one of the strongest predictive features we identified. The almost 2x difference in lead time between canceled and non-canceled bookings suggests this should be weighted heavily in our models.

Categorical Features Analysis

Market Segment Impact

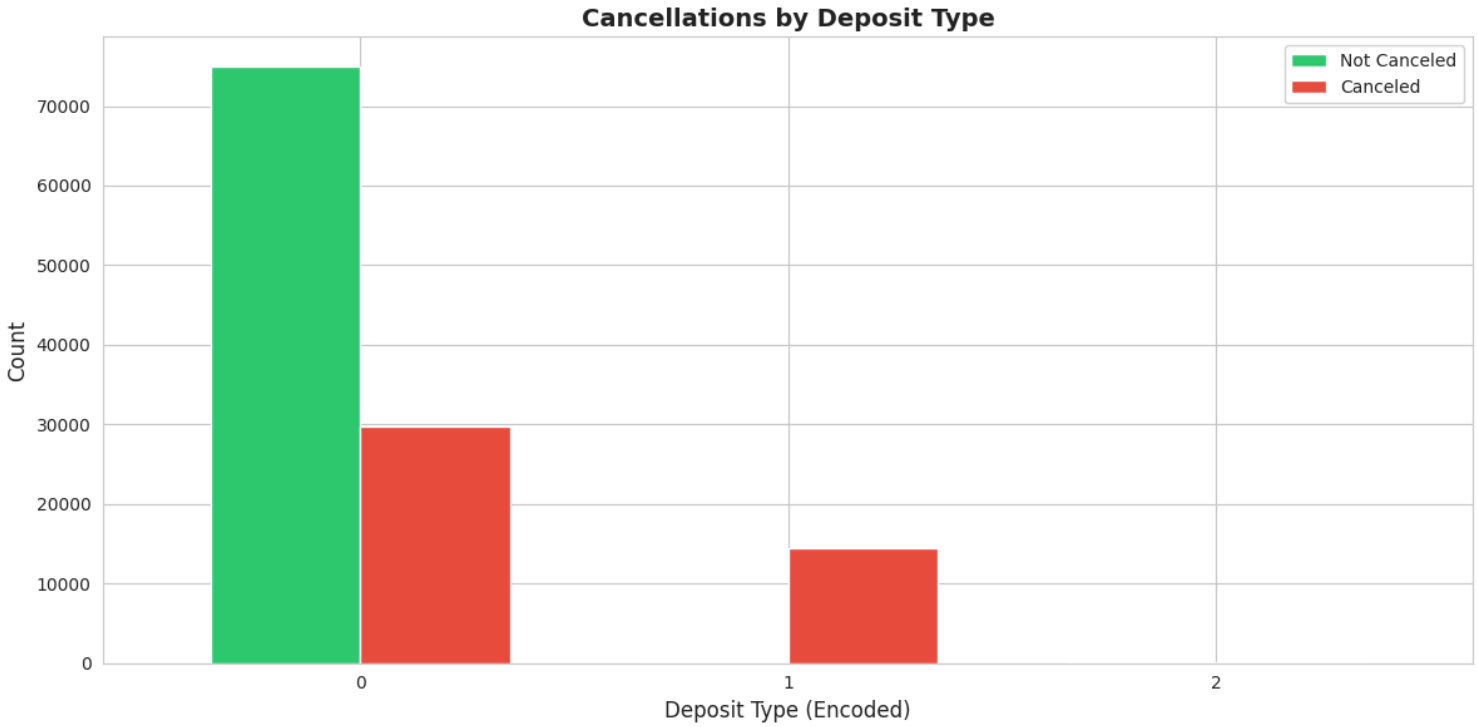


Cancellation Rate by Market Segment:

- Segment 7: 100.00% (always cancels - very small sample)
- Segment 4: 61.06%
- Segment 6: 36.72%
- Segment 1: 13.06% (most reliable)

Different market segments show dramatically different cancellation behaviors. This suggests that the booking channel or customer segment type is a critical predictor. Business segments typically show lower cancellation rates than leisure segments.

Customer Type Analysis



Cancellation Rate by Customer Type:

- Customer Type 2: 40.75%
- Customer Type 0: 30.96%
- Customer Type 1: 10.23% (most reliable)

Transient customers (likely one-time visitors) show higher cancellation rates compared to regular or group customers. This encoded feature captures customer loyalty and commitment levels.

Deposit Type - The Strongest Predictor



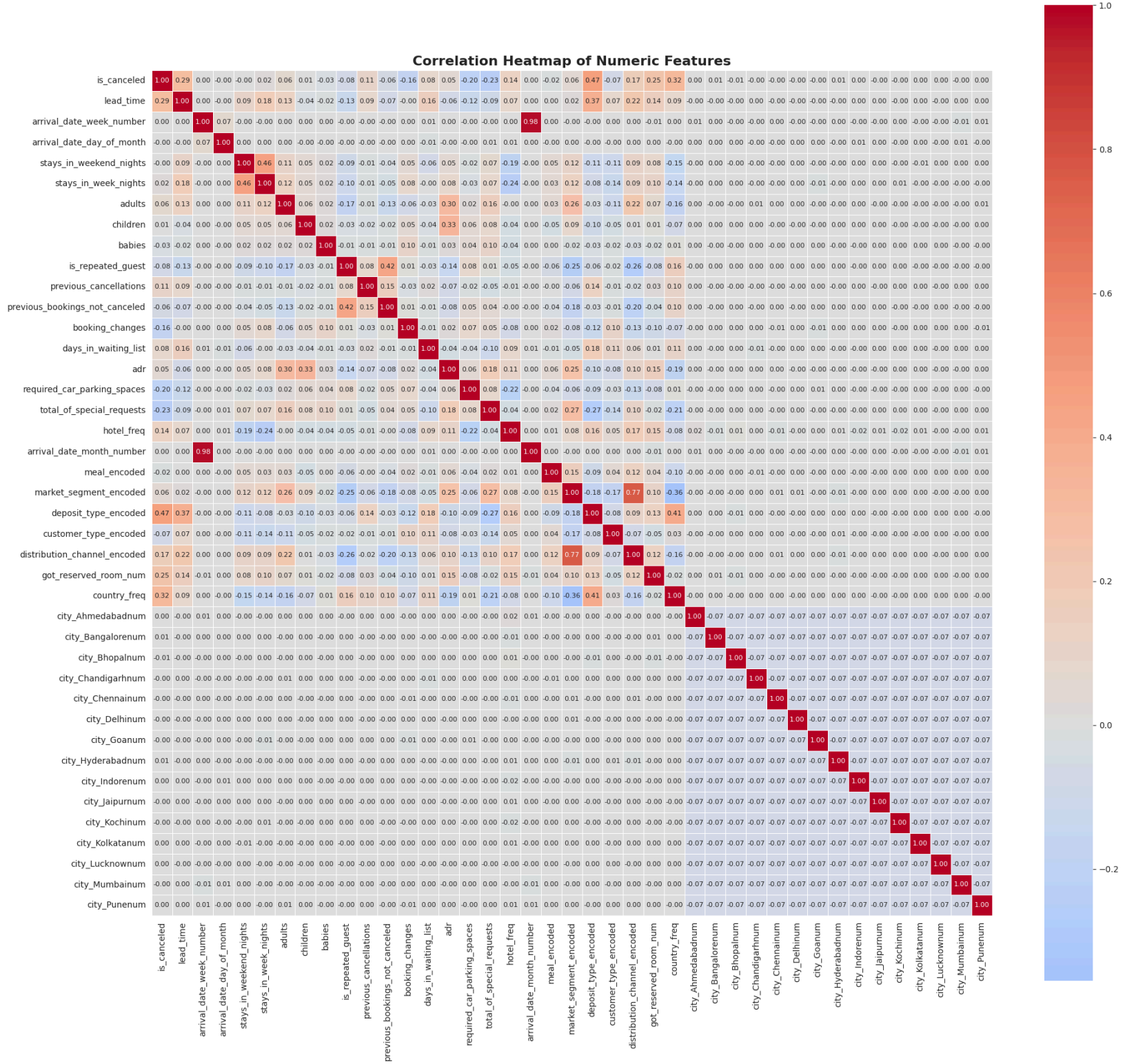
Cancellation Rate by Deposit Type:

- Deposit Type 1: 99.36% (almost always cancels)
- Deposit Type 0: 28.38%
- Deposit Type 2: 22.22%

Critical Finding: Deposit type emerged as THE strongest indicator of cancellation likelihood. When a certain deposit type is required (Type 1), cancellations are almost guaranteed. This makes perfect sense from a business perspective - certain deposit policies might be applied to risky bookings or might actually create an incentive to cancel.

This feature alone could drive much of our model's predictive power.

Correlation Analysis



The correlation heatmap reveals the relationships between features and helps identify multicollinearity issues.

Top Positive Correlations with Cancellation:

1. deposit_type_encoded: 0.469
2. country_freq: 0.319
3. lead_time: 0.294
4. got_reserved_room_num: 0.248
5. distribution_channel_encoded: 0.168

Top Negative Correlations with Cancellation:

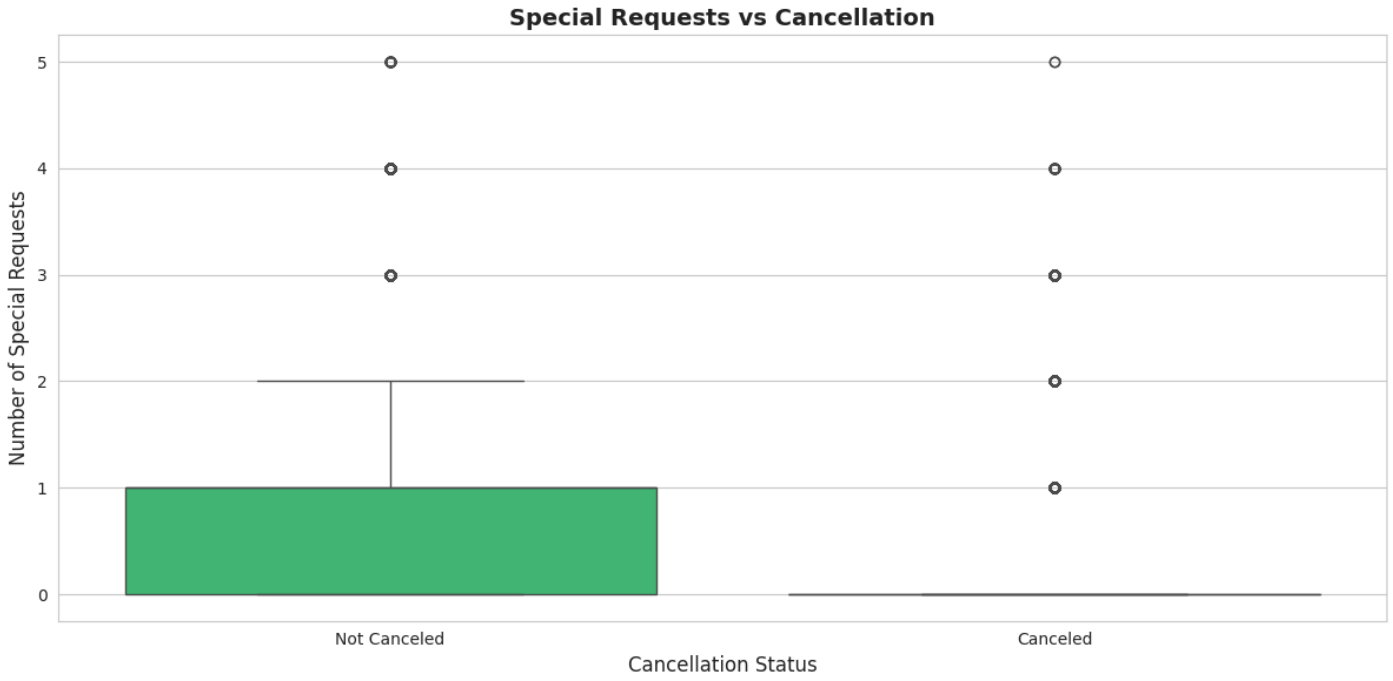
1. total_of_special_requests: -0.235 (most protective)
2. required_car_parking_spaces: -0.195
3. booking_changes: -0.161
4. is_repeated_guest: -0.085
5. customer_type_encoded: -0.068

Concluded:

- Features with positive correlation increase cancellation likelihood
- Features with negative correlation protect against cancellations
- Special requests show customers are engaged and committed
- Repeat guests are more reliable
- Parking space requirements indicate serious travel plans

Customer Engagement Features

Special Requests Impact

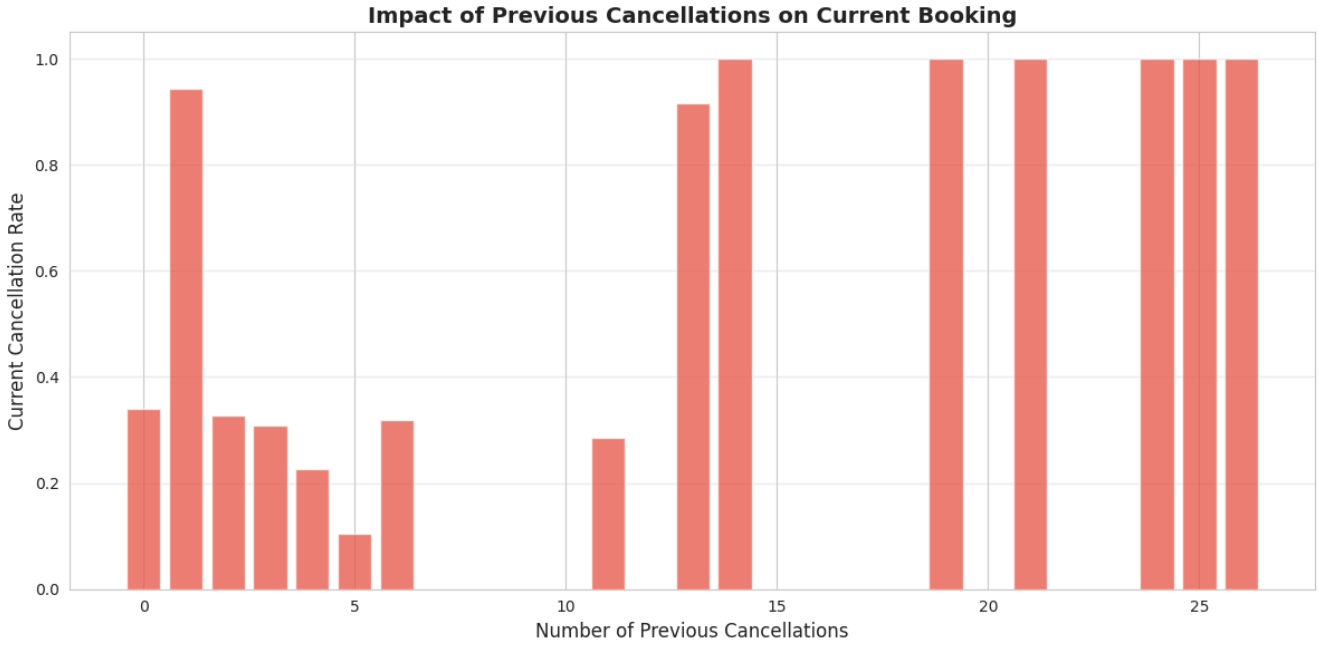


Findings:

- **Mean Special Requests for Not Canceled:** 0.71
- **Mean Special Requests for Canceled:** 0.33

Bookings with more special requests have lower cancellation rates. This indicates customer commitment and engagement with their booking. When customers take time to specify room preferences, dietary requirements, or other special needs, they're demonstrating serious intent to complete their stay.

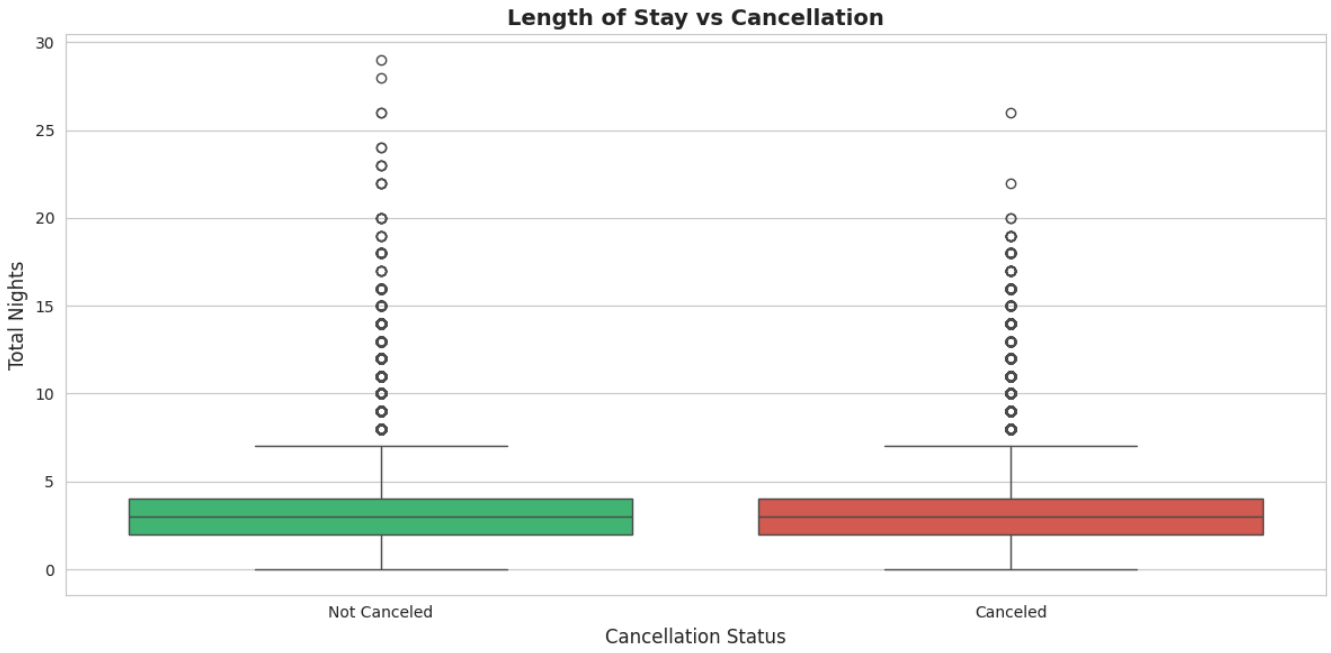
Previous Cancellation History



Customers with a history of cancellations are MORE likely to cancel again. This behavioral pattern is a strong predictor - past behavior predicts future behavior. The cancellation rate increases dramatically as the number of previous cancellations grows.

This historical feature should be weighted significantly in our models as it captures customer reliability patterns.

Length of Stay Analysis



Findings:

- **Mean Total Stay for Not Canceled:** 3.38 nights
- **Mean Total Stay for Canceled:** 3.46 nights

While the difference is small, longer stays tend to have slightly higher cancellation rates. This could be because longer commitments give more opportunity for plans to change.

Understanding Class Imbalance

Before building any machine learning models, we need to address a critical issue that can significantly impact model performance: class imbalance. Class imbalance occurs when one class in the target variable has significantly more samples than the other. This is a common problem in real-world datasets, especially in scenarios like fraud detection, disease diagnosis, or in our case, booking cancellations.

Why Class Imbalance Matters

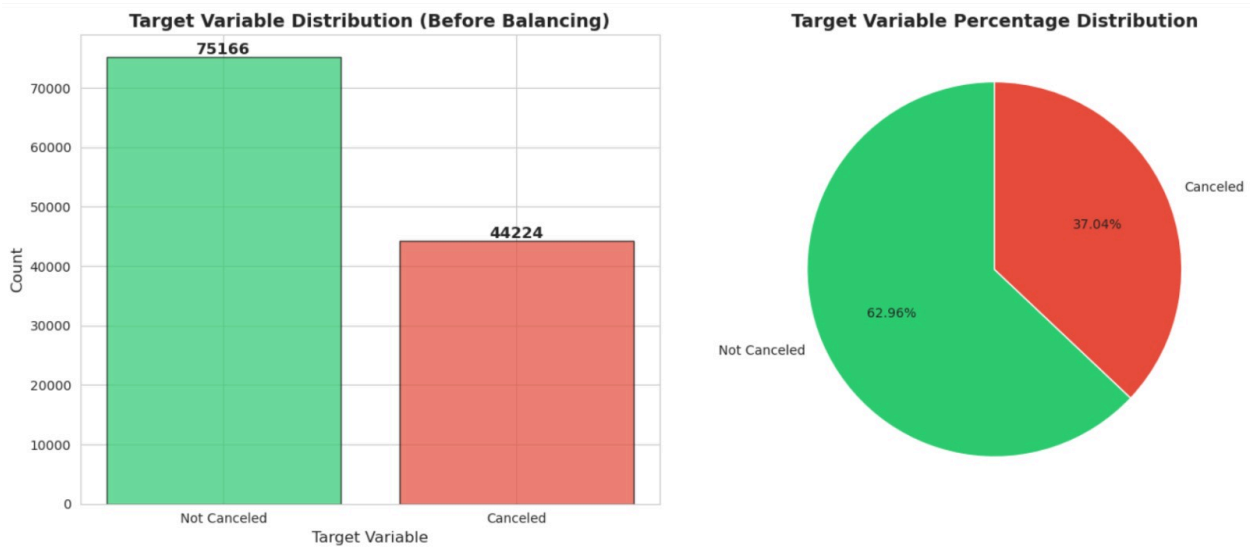
When a dataset is imbalanced, machine learning models can develop a bias toward the majority class. For example, if 95% of bookings are not canceled, a model could achieve 95% accuracy by simply predicting "not canceled" for every booking. While this looks good on paper, it would be completely useless for predicting actual cancellations, which is what we actually care about.

In our hotel booking scenario, identifying cancellations is more valuable than identifying non-cancellations. Hotels need to know which bookings are at risk so they can:

- Overbook strategically to compensate for expected cancellations
- Reach out to customers proactively to confirm bookings
- Adjust pricing and inventory management
- Prepare backup plans for last-minute cancellations

Current Dataset Balance

Let's examine our target variable distribution in detail.



From our earlier analysis, we know:

- **Total Bookings:** 119,390
- **Not Canceled (Class 0):** 75,166 samples (62.96%)
- **Canceled (Class 1):** 44,224 samples (37.04%)
- **Imbalance Ratio:** 1.70:1 (minority to majority)

Is Our Dataset Imbalanced?

The answer is: ***moderately imbalanced, but manageable.***

Here's why:

- An imbalance ratio of 1.70:1 is not severe. Severe imbalance would be something like 10:1 or 50:1.
- The minority class (canceled bookings) still represents 37% of the data, which is substantial.
- Many real-world classification problems work well with this level of imbalance.

However, we still need to be cautious because:

- Models might still show bias toward the majority class
- Performance metrics like accuracy alone won't tell the full story
- Some algorithms (especially neural networks) can be sensitive even to moderate imbalance

Evaluation Strategy for Imbalanced Data

Before deciding whether to apply resampling techniques, we need to understand how to properly evaluate models on imbalanced data.

Why Accuracy Is Not Enough

Consider this scenario:

- Our baseline (always predicting "not canceled") would give us 63% accuracy
- But it would have 0% recall for the minority class (canceled bookings)
- This makes the model completely useless for our business problem

Better Metrics for Imbalanced Data

We'll focus on these metrics:

Precision: Of all the bookings we predict will cancel, what percentage actually cancels?

- Important because false alarms waste hotel resources
- Formula: True Positives / (True Positives + False Positives)

Recall (Sensitivity): Of all the actual cancellations, what percentage do we correctly identify?

- Most critical metric for our use case
- Formula: True Positives / (True Positives + False Negatives)

F1-Score: Harmonic mean of precision and recall

- Balances both metrics
- Especially useful for imbalanced datasets
- Formula: 2 × (Precision × Recall) / (Precision + Recall)

Confusion Matrix: Shows the breakdown of all four prediction outcomes

- True Positives (TP): Correctly predicted cancellations
- True Negatives (TN): Correctly predicted non-cancellations
- False Positives (FP): Wrongly predicted cancellations (Type I error)
- False Negatives (FN): Missed cancellations (Type II error)

For our hotel booking problem, **missing a cancellation (FN) is worse than a false alarm (FP)**. It's better to be cautious and prepare for potential cancellations than to be caught off guard.

Oversampling with SMOTE

SMOTE (Synthetic Minority Over-sampling Technique)

Is the most sophisticated approach.

How SMOTE Works:

1. For each minority class sample, find its k nearest neighbors (typically k=5)
2. Randomly select one of these neighbors
3. Create a synthetic sample along the line connecting the two samples
4. Repeat until classes are balanced

Pros:

- Creates realistic synthetic samples (not just duplicates)
- Preserves all original data
- Proven effective for imbalanced classification
- Works well with our dataset size

Cons:

- Increases dataset size (more training time)
- Can create noise if minority class clusters are sparse
- May lead to overfitting if not careful

Strategic Approach

Strategy 1: Train models WITHOUT SMOTE first (Baseline)

- Use the original distribution
- Apply class weights where applicable
- Evaluate using precision, recall, and F1-score
- See if the 37% minority class is sufficient

Strategy 2: Train models WITH SMOTE (Comparison)

- Apply SMOTE only to the training set
- Keep validation and test sets untouched (critical for honest evaluation)
- Compare performance against baseline
- Determine if synthetic sampling helps

Why both approaches? Our 37% minority class might be enough for models to learn patterns effectively. SMOTE could help, but it also might introduce noise or overfitting. By trying both approaches, we can make a data-driven decision about which works better for our specific case.

Implementing SMOTE

- SMOTE should NEVER be applied before splitting the data
- Validation and test sets must remain original, real data
- Applying SMOTE before splitting causes data leakage

```
# 1. Split data first

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 2. Apply SMOTE only to training data

smote = SMOTE(random_state=42) X_train_balanced, y_train_balanced =
smote.fit_resample(X_train, y_train)

# 3. Train model on balanced training data

model.fit(X_train_balanced, y_train_balanced)

predictions = model.predict(X_test)
```

SMOTE Application Results

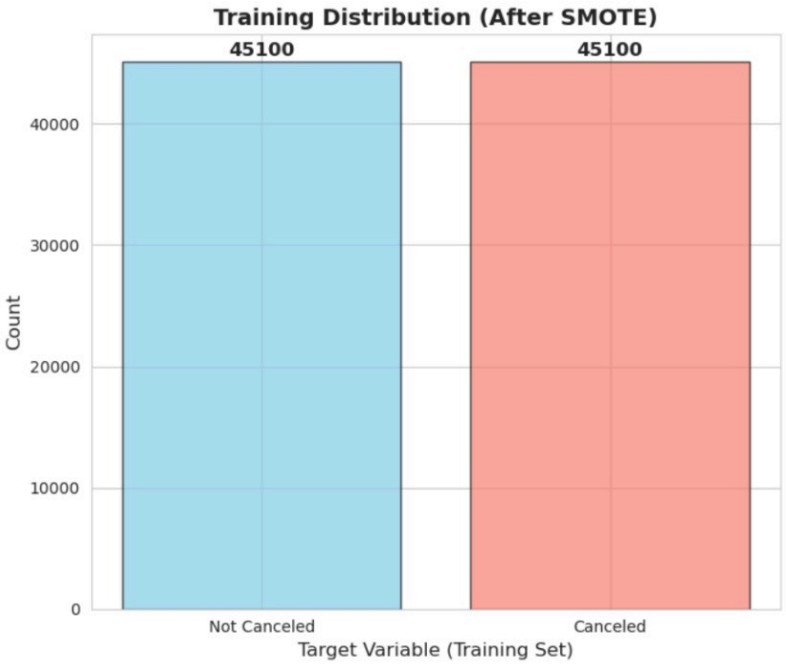
After applying SMOTE to our training set:

Before SMOTE (Original Training Data):

- Training samples: 71,634
- Not Canceled: ~45,105 (63%)
- Canceled: ~26,529 (37%)
- Imbalance Ratio: 1.70:1

After SMOTE (Balanced Training Data):

- Training samples: 90,210
- Not Canceled: 45,105 (50%)
- Canceled: 45,105 (50%)
- Imbalance Ratio: 1:1





Validation and Test Sets Remain Unchanged

This is crucial - our validation and test sets maintain the original distribution:

Validation Set:

- Total: 23,878 samples
- Not Canceled: ~15,034 (63%)
- Canceled: ~8,844 (37%)

Test Set:

- Total: 23,878 samples
- Not Canceled: ~15,034 (63%)
- Canceled: ~8,844 (37%)

By keeping these sets in their natural state, we ensure that our performance metrics reflect real-world expectations. When we deploy this model, it will encounter the natural 37% cancellation rate, not an artificial 50% rate.

How SMOTE Helps

For training:

- Models see equal representation of both classes
- Learning algorithms don't develop majority class bias
- More examples of minority class patterns
- Better feature importance estimation for cancellation predictors

Specifically helps:

- Neural networks (sensitive to class balance)
- Distance-based algorithms like KNN (benefit from more minority samples)
- Decision trees (less affected but still benefit)

Feature Engineering

To improve the predictive capability of the machine learning models, additional features were engineered by combining and transforming existing attributes. These features were designed to better represent booking behavior and customer characteristics that may influence cancellation decisions.

Total Stay Duration

```
df['total_stay'] = df['stays_in_weekend_nights'] +
                   df['stays_in_week_nights']
```

This feature represents the total number of nights booked by combining weekday and weekend stays. The total length of stay provides a clearer indicator of booking commitment than treating weekday and weekend nights separately.

Total Number of Guests

```
df['total_guests'] = df['adults'] + df['children'] +
                     df['babies']
```

This feature aggregates all guest types into a single value. The total number of guests helps capture booking size, which can affect pricing, room allocation, and cancellation behavior.

Family Booking Indicator

```
df['is_family'] = ((df['children'] + df['babies']) >
                   0).astype(int)
```

A binary feature was created to indicate whether a booking involves a family (i.e., includes children or babies). Family bookings may have different cancellation patterns compared to bookings made by adults only.

Average Daily Rate (ADR) Level

```
df['adr_level'] = pd.qcut(df['adr'], q=3, labels=[0, 1,
                    2]).astype(int)
```

The average daily rate (ADR) was converted into three categorical levels (low, medium, high) using quantile-based discretization. This transformation reduces the effect of extreme values and allows the model to learn pricing-related patterns more effectively.

MODEL BUILDING

DATA SPLITTING

before building or designing the models, proper prep is essential; here's our pipeline>
data splitting > imbalance handling > feature scaling > model training

Let's get to it, what is data splitting?

The concept of data splitting is the fundamental portioning of a dataset into subsets of different learning workflow; lean, validate and test.

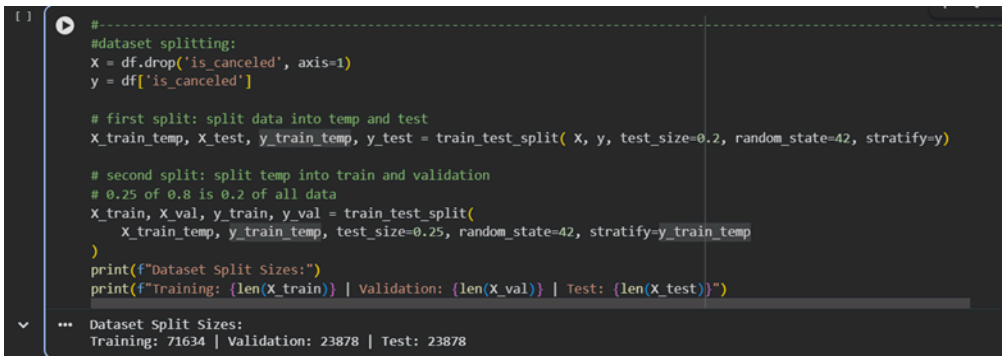
Why does this work?

It prevents data leakage , and provides unbiased evaluation of the model performance.

We follow a three way split as models need to

1. learn patterns
2. Validate decisions during development
3. Test generalization on unseen data

IMPLEMENTATION:



```
#----------------------------------------------------------------------------------------------------------------------------------
#dataset splitting:
X = df.drop('is_canceled', axis=1)
y = df['is_canceled']

# first split: split data into temp and test
X_train_temp, X_test, y_train_temp, y_test = train_test_split( X, y, test_size=0.2, random_state=42, stratify=y)

# second split: split temp into train and validation
# 0.25 of 0.8 is 0.2 of all data
X_train, X_val, y_train, y_val = train_test_split(
    X_train_temp, y_train_temp, test_size=0.25, random_state=42, stratify=y_train_temp
)
print(f"Dataset Split Sizes:")
print(f"Training: {len(X_train)} | Validation: {len(X_val)} | Test: {len(X_test)}")

Dataset Split Sizes:
Training: 71634 | Validation: 23878 | Test: 23878
```

The code above is our complete implementation for splitting; here's how it works:

Step 1 (below dataset splitting text) creates feature matrix x by removing the target column; axis =1 means drop column.

What this does is keep all booking characteristics except cancellation status
we then create a target vector y containing only cancellation status.

Then we use train_test_split to input our dataset, with the output being 20% locked away , 80% for further splitting and stratify ensures the test set has the same cancellation rate as OG. (37%)

The second function inputs the 80% temp dataset; outputs 60% of original data for model training, 20% for validation, and keeps test_size = 0.25 which remains 20% of data.

then we print out dataset sizes; as portrayed in the screenshot above.

1. SMOTE (handling class imbalance

In theory, class imbalance occurs when one class is significantly under-represented compared to another class; in hotel booking data typically only 30% of bookings get canceled, creating a skewed distribution.

This causes an imbalance problem as it can bias learning models, models tend to favor the majority class and predict not canceled most often; leading to an unfair and biased decision making process.

We have a few approaches to handle this:
undersampling > removes majority class samples (loses info)

Oversampling > duplicate minority class samples (overfitting)
SMOTE > create new, similar minority samples. (optimal)

How it works:

SMOTE finds its K-nearest neighbors, randomly selects one and creates a synthetic sample along a line connecting original and neighbor.

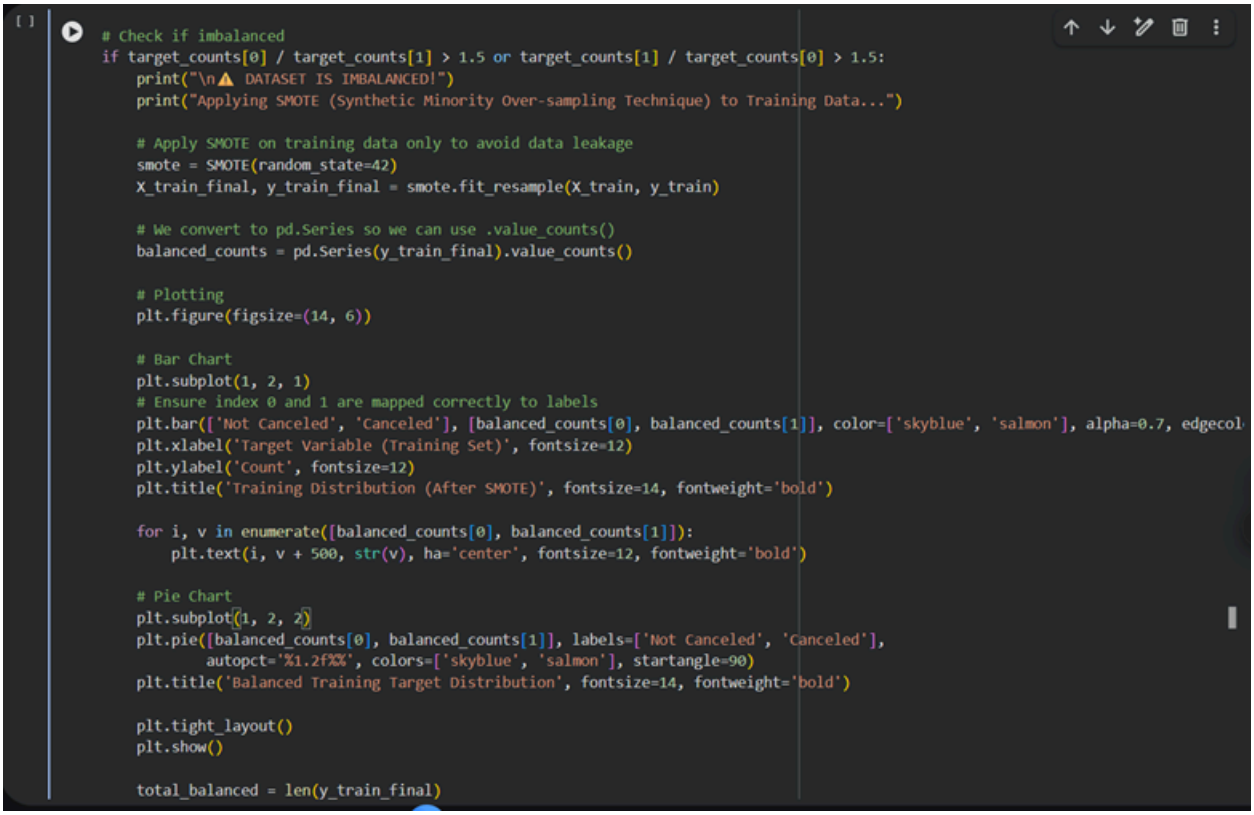
new_sample = original + λ × (neighbor - original)

where $\lambda \in [0, 1]$ is a random number

Key Advantages:

- Creates diverse synthetic samples (not just duplicates)
- Expands decision boundary region for minority class
- Avoids exact replication that causes overfitting

PRACTICAL IMPLEMENTATION:



```
# Check if imbalanced
if target_counts[0] / target_counts[1] > 1.5 or target_counts[1] / target_counts[0] > 1.5:
    print("\n⚠ DATASET IS IMBALANCED!")
    print("Applying SMOTE (Synthetic Minority Over-sampling Technique) to Training Data...")

    # Apply SMOTE on training data only to avoid data leakage
    smote = SMOTE(random_state=42)
    X_train_final, y_train_final = smote.fit_resample(X_train, y_train)

    # We convert to pd.Series so we can use .value_counts()
    balanced_counts = pd.Series(y_train_final).value_counts()

    # Plotting
    plt.figure(figsize=(14, 6))

    # Bar Chart
    plt.subplot(1, 2, 1)
    # Ensure index 0 and 1 are mapped correctly to labels
    plt.bar(['Not Canceled', 'Canceled'], [balanced_counts[0], balanced_counts[1]], color=['skyblue', 'salmon'], alpha=0.7, edgecol
    plt.xlabel('Target Variable (Training Set)', fontsize=12)
    plt.ylabel('Count', fontsize=12)
    plt.title('Training Distribution (After SMOTE)', fontsize=14, fontweight='bold')

    for i, v in enumerate([balanced_counts[0], balanced_counts[1]]):
        plt.text(i, v + 500, str(v), ha='center', fontsize=12, fontweight='bold')

    # Pie Chart
    plt.subplot(1, 2, 2)
    plt.pie([balanced_counts[0], balanced_counts[1]], labels=['Not Canceled', 'Canceled'],
            autopct='%1.2f%%', colors=['skyblue', 'salmon'], startangle=90)
    plt.title('Balanced Training Target Distribution', fontsize=14, fontweight='bold')

    plt.tight_layout()
    plt.show()

    total_balanced = len(y_train_final)
```

```
        total_balanced = len(y_train_final)
        print(f"\n✅ BALANCED TRAINING CLASS DISTRIBUTION:")
        print(f"Not Canceled (0): {balanced_counts[0]} ({balanced_counts[0]/total_balanced*100:.2f}%)")
        print(f"Canceled (1): {balanced_counts[1]} ({balanced_counts[1]/total_balanced*100:.2f}%)")
        print(f"New Training Imbalance Ratio: {balanced_counts[0]/balanced_counts[1]:.2f}:1")

        print("\nNOTE: The Test Set remains original and imbalanced for realistic evaluation.")

    else:
        print("\nDATASET IS REASONABLY BALANCED!")
        X_train_final, y_train_final = X_train, y_train
```

What this does, a line by line analysis:

1. We do imbalance detection using target_counts, which is an array containing counts of class 0 and 1. If ratio > 1.5 SMOTE is triggered (either class is 50% more prevalent)

2. For no leakage we set up SMOTE application, smote(random_state=42) initializes with reproducibility seed; fit resample fits on training data only, then resamples by creating synthetic examples.

3. Balanced distribution analysis happens which then turns the data from 60:40 ratio to a perfect 50:50 ratio of 86,000 samples (14,366 added synthetic)

Then we do visualizations using a barchart, pie chart, and print out a statistical summary output.

BALANCED TRAINING CLASS DISTRIBUTION:

Not Canceled (0): 43000 (50.00%)

Canceled (1): 43000 (50.00%)

New Training Imbalance Ratio: 1.00:1

DECISION TREE CLASSIFIER

In theory, a decision tree is a tree classification model that predicts by learning simple decision rules inferred from data features. It's a white box model where the prediction process is completely transparent and can be interpreted.

How Decision Trees Work:

1. Root Node: Starts with entire dataset
2. Splitting: Recursively partitions data based on feature values
3. Decision Nodes: Internal nodes that test specific features
4. Leaf Nodes: Final predictions (cancelled/not cancelled)

Why Does It Work?
interpretable: hotel managers can understand how it works and the percentage of cancellation it predicts.
handles mixed data: works with both numeric and categorial features.

No scaling needed compared to KNN and MLP.

Feature importance : naturally reveals which booking attributes most affect cancellations.

PRACTICAL IMPLEMENTATION:

```
# Decision Tree with all features
#Initialize model
dt_all = DecisionTreeClassifier(random_state=42)

# Training
dt_all.fit(X_train_final, y_train_final)

# Validation (It's like a pretend test that shows us results that allow us to tune hyperparameters)
y_pred_dt_val_all = dt_all.predict(X_val)
acc_dt_val_all = accuracy_score(y_val, y_pred_dt_val_all)
f1_dt_val_all = f1_score(y_val, y_pred_dt_val_all)
print(f"--- Decision Tree Baseline Validation ---")
print(f"Validation Accuracy: {acc_dt_val_all:.4f}")
print(f"Validation F1-Score: {f1_dt_val_all:.4f}")

#Testing
y_pred_dt_all = dt_all.predict(X_test)
acc_dt_all = accuracy_score(y_test, y_pred_dt_all)
f1_dt_all = f1_score(y_test, y_pred_dt_all)
print(f"\n--- Decision Tree Baseline Test ---")
print(f"Test Accuracy: {acc_dt_all:.4f}")
print(f"Test F1-Score: {f1_dt_all:.4f}")
```

```
--- Decision Tree Baseline Validation ---
Validation Accuracy: 0.8044
Validation F1-Score: 0.7428

--- Decision Tree Baseline Test ---
Test Accuracy: 0.8058
Test F1-Score: 0.7432
```

A. All features:

1. Initialization

Default parameters use gini impurity, unlimited depth until pure leaves; then random_state=42 ensures reproducible tree structure and no explicit parameters.

2. Model training

X_train_final input is balanced smote data whilst y_train_final is balanced labels

It recursively splits the data to minimize impurity

Output > trained tree structure stored in dt_all

3. Validation evaluation

The validation set acts as a proxy test set during development. We use it to:

Check if model generalizes (not overfitting)

Compare different models/parameters

Make development decisions

Interpretation:

correctly classifies 80.44% of validation bookings

74.28% F1 score: balanced measure considering both precision and recall

4. Final test eval

Correctly classifies 80.58% of validation bookings, and an F1 score of 75.32%

Small difference (<0.5%) suggests good generalization.

B. Genetic Algorithm

```python
# GA 1: using decision trees
dt_model = DecisionTreeClassifier(random_state=42)

ga_dt = GAFeatureSelectionCV(
    estimator=dt_model,
    cv=5,
    scoring="accuracy",
    population_size=50,
    generations=40,
    crossover_probability=0.5,
    mutation_probability=0.2,
    n_jobs=-1,
    verbose=True
)

ga_dt.fit(X_train_final, y_train_final)

selected_features_dt = X.columns[ga_dt.support_]
print("DT Selected Features:", list(selected_features_dt))

# Training
dt_model.fit(X_train_final[selected_features_dt], y_train_final)

# Validation
y_pred_dt_val = dt_model.predict(X_val[selected_features_dt])
print("\n--- Decision tree validation report ---")
print(f"Validation Accuracy: {accuracy_score(y_val, y_pred_dt_val):.4f}")
print(f"Validation F1-Score: {f1_score(y_val, y_pred_dt_val):.4f}")

# Testing
y_pred_dt = dt_model.predict(X_test[selected_features_dt])
acc_dt = accuracy_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt)
print("DT Accuracy after GA:", acc_dt)
print("--- Decision tree final report ---")
print(classification_report(y_test, y_pred_dt))
```

```python
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_dt))
```

gen	nevals	fitness	fitness_std	fitness_max	fitness_min
0	50	0.762937	0.0346433	0.820189	0.664882

GA process:

Evolutionary Steps:

1. Initialization: Create 50 random feature subsets (population)
2. Evaluation: Test each subset using 5-fold cross-validation accuracy
3. Selection: Keep best-performing subsets ("survival of the fittest")

4. Crossover: Combine features from parent subsets (50% probability)

5. Mutation: Randomly add/remove features (20% probability)

6. Repeat: 40 generations of evolution

Why GA for Feature Selection:

· 28! possible combinations = 304,888,344,000,000,000 possibilities

· GA finds good solutions in reasonable time

· Better than forward/backward selection for complex interactions

Business Interpretation:

- Precision (class 1) = 0.82: When model predicts cancellation, it's correct 82% of time
- Recall (class 1) = 0.74: Catches 74% of actual cancellations
- Hotel Action: Can confidently offer incentives to predicted cancellations (high precision), but misses 26% of actual cancellations

Business Impact Analysis:

- 1202 false positives: Offer unnecessary incentives to bookings that won't cancel ($ cost)
- 2302 false negatives: Miss actual cancellations (lost revenue cost)
- Trade-off: Balance depends on cost of incentive vs cost of empty room

3.SCALING

What is scaling? Feature scaling is the process of normalizing the range of variables so they have similar scales. This is critical for distance-based algorithms like KNN so they must be done beforehand. But not needed for tree based like DT.

Without scaling, features with larger ranges will dominate most distance calculations

distorting the models learning process.

When choosing our method we went with standardization; which centers data around mean (0) , scales to unit variance (1). Best for normally distributed data. Works well regardless.

PRACTICAL IMPLEMENTATION

```
# scaling data (necessary for KNN and MLP)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_final)
# we also scale the validation and test splits
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

First, we initialized the scaler object that will calculate mean and std from training data; and store these parameters for consistent transformation. The formula was (x-mean)/std.

Secondly, we fit the training data:
fit(): learns the parameters from the training data and calculates
mean_train = mean(X_train_final)
std_train=standard_deviation(X_train_final) and stores these values.

Transform(): applies the transformation for each value using the previously mentioned formula.

X_train_scaled has mean=0 , std=1 for each feature.

NOTICE: we used transform only not fit_transform since it has a fixed mean and std equation, which avoids data leakage and model seeing info from validation/test during training.

KNN

To start we need to understand what KNN is, KNN is a distance-based classification algorithm that works on the assumption that things similar will exist closer in proximity to one another. So when applying this to hotel cancellation we should understand that: Booking with similar characteristics (deposit type for example) should have similar cancellation outcomes.

Advantages of this?

It can be simple and intuitive, no assumption about data distribution, it could work as patterns often do cluster, such that travelers book differently from families, and last minute bookings behave differently from advance ones.

Drawbacks: sensitive to feature scaling, computationally expensive with large datasets.

PHYSICAL IMPLEMENTATION:

```python
# KNN baseline with all features

#Initialize KNN model
knn_all = KNeighborsClassifier(n_neighbors=5, weights='distance')

# Training
knn_all.fit(X_train_scaled, y_train_final)

# Validation
y_pred_knn_val_all = knn_all.predict(X_val_scaled)
acc_knn_val_all = accuracy_score(y_val, y_pred_knn_val_all)
f1_knn_val_all = f1_score(y_val, y_pred_knn_val_all)
print(f"--- KNN Baseline Validation ---")
print(f"Validation Accuracy: {acc_knn_val_all:.4f}")
print(f"Validation F1-Score: {f1_knn_val_all:.4f}")

#Testing
y_pred_knn_all = knn_all.predict(X_test_scaled)
acc_knn_all = accuracy_score(y_test, y_pred_knn_all)
f1_knn_all = f1_score(y_test, y_pred_knn_all)
print(f"\n--- KNN Baseline Test ---")
print(f"Test Accuracy: {acc_knn_all:.4f}")
print(f"Test F1-Score: {f1_knn_all:.4f}")
```

```
--- KNN Baseline Validation ---
Validation Accuracy: 0.7795
Validation F1-Score: 0.7019

--- KNN Baseline (All Features) Test ---
Test Accuracy: 0.7802
Test F1-Score: 0.6990
```

Now let's elaborate on this:

1. Model initialization

What knn_all does is create a classifier that looks at 5 nearest neighbors and gives weight accordingly, with more the closer the neighbors.

2. Training

We then “train” the model using scaled training data. Which should be 71,634 bookings after SMOTE

We do this in order to teach the model how bookings with similar scaled features tend to have similar cancellation outcomes.

3. Validation evaluation

What this does is predict cancellations for the validation set; and calculates the accuracy and f1_score.

This is the model's “practice test” basically. That’s the last text before touching the real test. Then ..

4. The final test evaluation

This is the true performance measurement - how well the model works on completely new data it's never seen before.

Key Insight: The small drop from validation to test (<0.5%) shows the model generalizes well and isn't overfitting to the validation set.

Now, for the GA section:

```
# GA 2: using KNN
knn_model = KNeighborsClassifier(n_neighbors=5, weights='distance')

ga_knn = GAFeatureSelectionCV(
    estimator=knn_model,
    cv=5,
    scoring="accuracy",
    population_size=30,
    generations=20,
    crossover_probability=0.5,
    mutation_probability=0.2,
    n_jobs=-1,
    verbose=True
)

ga_knn.fit(X_train_scaled, y_train_final)

selected_features_knn = X.columns[ga_knn.support_]  #[:, ga_knn.support_] is a b
print("KNN Selected Features:", list(selected_features_knn))

# Training
knn_model.fit(X_train_scaled[:, ga_knn.support_], y_train_final)

# Validation
y_pred_knn_val = knn_model.predict(X_val_scaled[:, ga_knn.support_])
print("\n--- KNN validation report ---")
print(f"Validation Accuracy: {accuracy_score(y_val, y_pred_knn_val):.4f}")
print(f"Validation F1-Score: {f1_score(y_val, y_pred_knn_val):.4f}")

# Testing
y_pred_knn = knn_model.predict(X_test_scaled[:, ga_knn.support_])
acc_knn = accuracy_score(y_test, y_pred_knn)
f1_knn = f1_score(y_test, y_pred_knn)
print("KNN Accuracy after GA:", acc_knn)
```

```
print("--- KNN final report ---")
print(classification_report(y_test, y_pred_knn))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_knn))
```

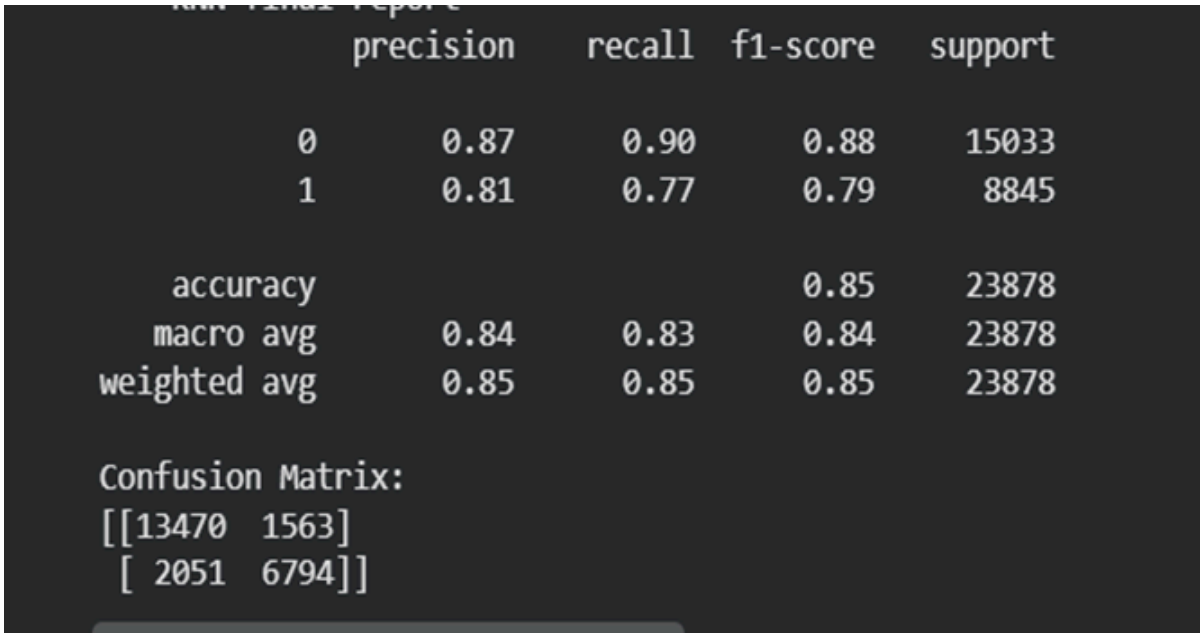
First thing we did was set up a GA that will evolve the best feature subset for KNN. It creates 30 random combinations and then evaluates them with 5-fold cross validation, evolving them over 20 generations afterwards.

Why? This saves time from manually trying all feature combinations by automatically finding one through simulated evolution.

Then we run the GA on scaled training data to find best features, extracting them using ga_knn.support_ , this identifies the actually useful features for KNN.

We then train a new KNN model using these features; validating it with reduced features using the same feature subset as training leading on to the final test , we use the optimized feature subset to make final predictions, this improves real world performance.

OUTPUT:



This output presents a classification report for a binary model. The model achieved an overall accuracy of 85%. Class 0 has higher precision (87%) and recall (90%) than class 1 (81% and 77%, respectively), indicating the model is more reliable at identifying class 0. The weighted average F1-score is 0.85, reflecting good overall performance.

The confusion matrix shows the model correctly predicted 13,470 instances of class 0 and 6,794 of class 1, while misclassifying 1,563 class 0 cases as class 1 and 2,051 class 1 cases as class 0.

Solid overall performance.

4.Multi-Layer Perceptron (MLP)

An MLP is a class of feedforward artificial neural network, it consists of usually three layers or more, an input layer, one or more hidden layers and an output one.

Each node uses a non-linear activation to process info, it learns by adjusting the weights of the connections between notes through a process called backpropagation, which minimizes the diff of outputs, bringing us closer to the actual target.

MLP's structure allows us to learn complex, non-linear decision boundaries, making it a really flexible yet powerful model for classification and regression tasks.

PRACTICAL IMPLEMENTATION

```
# MLP baseline with all features

#Initialize model
mlp_all = MLPClassifier(hidden_layer_sizes=(100, 50), solver='adam', max_iter=1000,
                        early_stopping=True, activation='logistic', n_iter_no_change=20, random_state=42)

#Training
mlp_all.fit(X_train_scaled, y_train_final)

# Validation
y_pred_mlp_val_all = mlp_all.predict(X_val_scaled)
acc_mlp_val_all = accuracy_score(y_val, y_pred_mlp_val_all)
f1_mlp_val_all = f1_score(y_val, y_pred_mlp_val_all)
print(f"--- MLP Baseline Validation ---")
print(f"Validation Accuracy: {acc_mlp_val_all:.4f}")
print(f"Validation F1-Score: {f1_mlp_val_all:.4f}")

#Testing
y_pred_mlp_all = mlp_all.predict(X_test_scaled)
acc_mlp_all = accuracy_score(y_test, y_pred_mlp_all)
f1_mlp_all = f1_score(y_test, y_pred_mlp_all)
print(f"\n--- MLP Baseline Test ---")
print(f"Test Accuracy: {acc_mlp_all:.4f}")
print(f"Test F1-Score: {f1_mlp_all:.4f}")
```

We trained MLP using the entire set of input features without any selection.

How it works:

MLPclassifier is configured and initialized with > architecture (defines 2 hidden layers with 100 and 50 neurons, respectively.)
Optimizer: ‘adam’ is an efficient adaptive learning rate algorithm

Regularization: functions like early_stopping=true halt training if the validation score doesn’t improve for 20 epochs, to avoid overfitting.

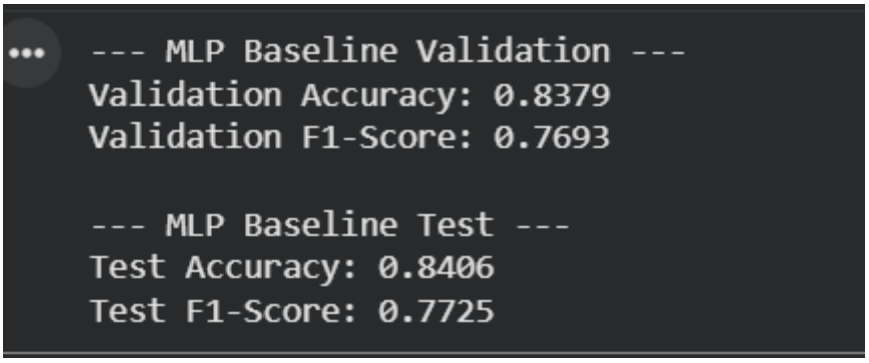
Non-linearity: activation = ‘logistic’ uses the sigmoid function

And we use random_state=42 to ensure reproducibility and consistent results.

Then, we train the model to learn the relationships between all scaled training features and target labels.

Finally, the evaluation; the trained model predicts outcomes on both a validation set and a “hold-out” test set; it reports accuracy and f1-score.

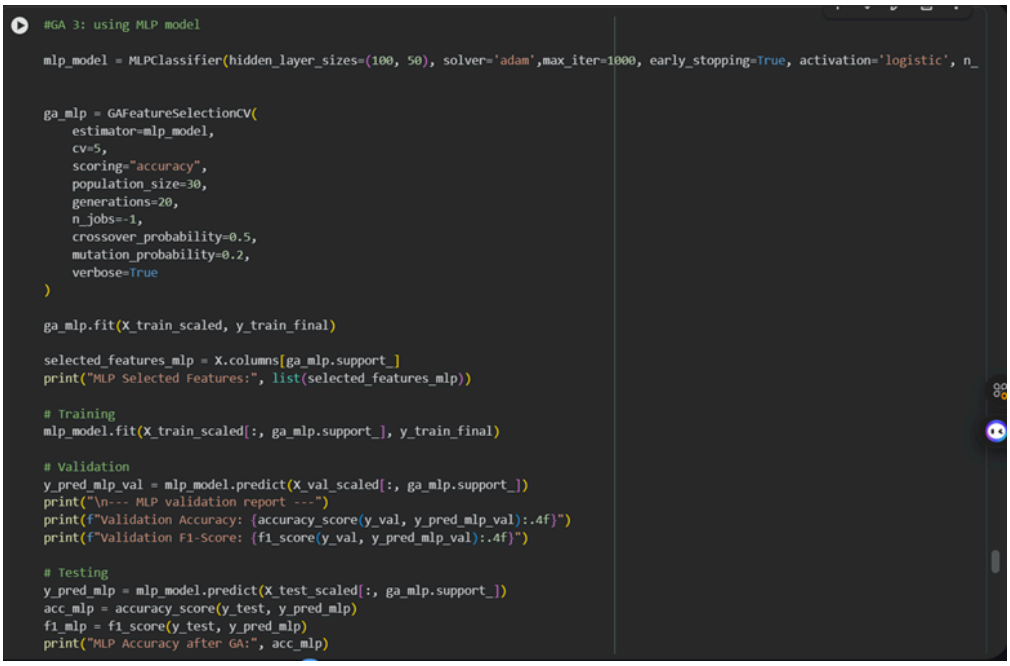
This code provides the benchmark performance of the MLP model. It shows how well the model can perform when given all available information, setting a reference point to judge whether subsequent feature selection improves or simplifies the model without significant performance loss.



Output shows the model is consistent between validation (0.8379 accuracy) and test

(0.8406 accuracy) sets, with no significant overfitting. The F1-scores (0.7693 and 0.7725) are lower than the accuracy, which is typical when class distribution is imbalanced—the model is better at predicting the majority class. Overall, these are the benchmark metrics to compare against the GA-optimized version.

Now for GA:



```
#GA 3: using MLP model

mlp_model = MLPClassifier(hidden_layer_sizes=(100, 50), solver='adam',max_iter=1000, early_stopping=True, activation='logistic', n_


ga_mlp = GAFeatureSelectionCV(
    estimator=mlp_model,
    cv=5,
    scoring="accuracy",
    population_size=30,
    generations=20,
    n_jobs=-1,
    crossover_probability=0.5,
    mutation_probability=0.2,
    verbose=True
)

ga_mlp.fit(X_train_scaled, y_train_final)

selected_features_mlp = X.columns[ga_mlp.support_]
print("MLP Selected Features:", list(selected_features_mlp))

# Training
mlp_model.fit(X_train_scaled[:, ga_mlp.support_], y_train_final)

# Validation
y_pred_mlp_val = mlp_model.predict(X_val_scaled[:, ga_mlp.support_])
print("\n--- MLP validation report ---")
print(f"Validation Accuracy: {accuracy_score(y_val, y_pred_mlp_val):.4f}")
print(f"Validation F1-Score: {f1_score(y_val, y_pred_mlp_val):.4f}")

# Testing
y_pred_mlp = mlp_model.predict(X_test_scaled[:, ga_mlp.support_])
acc_mlp = accuracy_score(y_test, y_pred_mlp)
f1_mlp = f1_score(y_test, y_pred_mlp)
print("MLP Accuracy after GA:", acc_mlp)
```



```
# Testing
y_pred_mlp = mlp_model.predict(X_test_scaled[:, ga_mlp.support_])
acc_mlp = accuracy_score(y_test, y_pred_mlp)
f1_mlp = f1_score(y_test, y_pred_mlp)
print("MLP Accuracy after GA:", acc_mlp)

print("--- MLP final report ---")
print(classification_report(y_test, y_pred_mlp))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_mlp))
```

The goal here is to improve the model efficiency and performance by identifying the most informative subset of features via GA.

How it works:

Feature Selection Wrapper: The GAFeatureSelectionCV object encapsulates the feature selection process:

- It uses the same mlp_model as its internal evaluator (estimator).
- It performs a Genetic Algorithm search over the space of possible feature subsets.

This evolutionary algorithm starts with a random "population" of feature subsets, evaluates their fitness (here, scoring="accuracy" via 5-fold cross-validation cv=5), and "breeds" the best ones over multiple generations.

- Parameters like population_size etc control the search behavior.

Then we enter the selection process where ga_mlp.fit runs this search on the training data. It identifies a binary mask (ga_mlp.support_) to indicate which features are selected.

Then we train and evaluate a new separate instance of the MLP model only on the selected features. It is validated and tested using data sliced with the same feature mask.

What it Achieves:

This approach performs automated feature engineering. It aims to find a smaller, optimal subset of features that maximizes the model's predictive power (accuracy). The potential benefits are:

- Improved Generalization: Removing irrelevant or noisy features can reduce overfitting.
- Increased Efficiency: Training and prediction become faster with fewer features.
- Enhanced Interpretability: A model built on 10 key features is easier to understand than one built on 100.

In summary, the first approach uses all data as-is to set a baseline, while the second uses an evolutionary algorithm to intelligently select the best features before building the final model.

OUTPUT:

```
--- MLP validation report ---
Validation Accuracy: 0.8413
Validation F1-Score: 0.7778
MLP Accuracy after GA: 0.8449200100510931
--- MLP final report ---
              precision    recall  f1-score   support

           0       0.86      0.90      0.88     15033
           1       0.82      0.75      0.78      8845

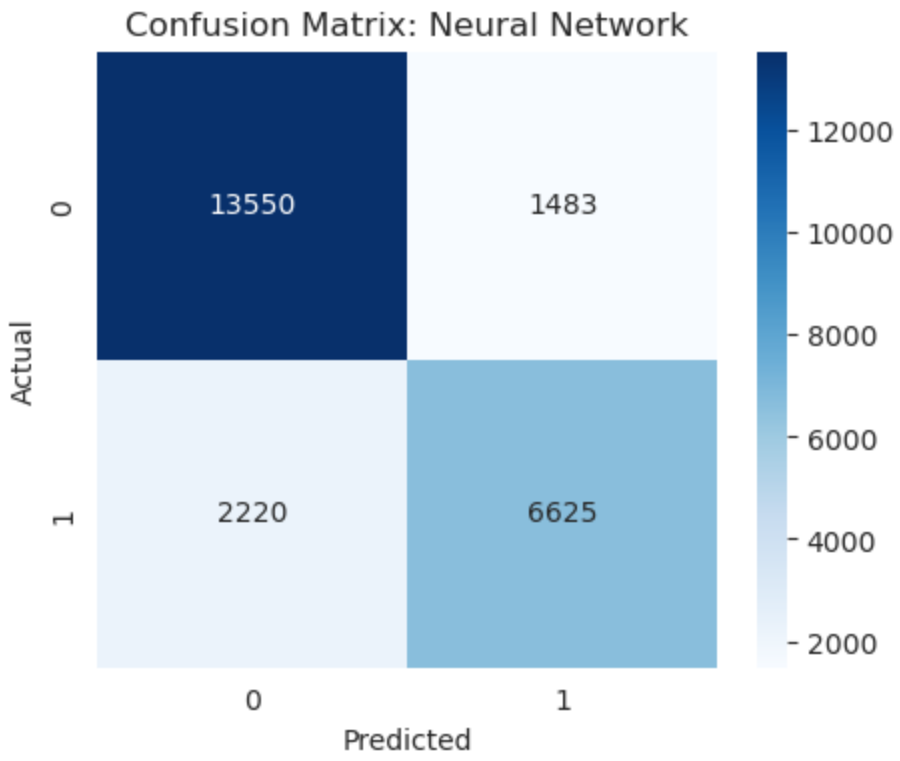
    accuracy                           0.84     23878
   macro avg       0.84      0.83      0.83     23878
weighted avg       0.84      0.84      0.84     23878

Confusion Matrix:
[[13550  1483]
 [ 2220  6625]]
```

The validation accuracy (0.8413) and F1-score (0.7778) improved slightly over the baseline. The final test accuracy reached ~0.845, which is a small improvement over the baseline's 0.8406.

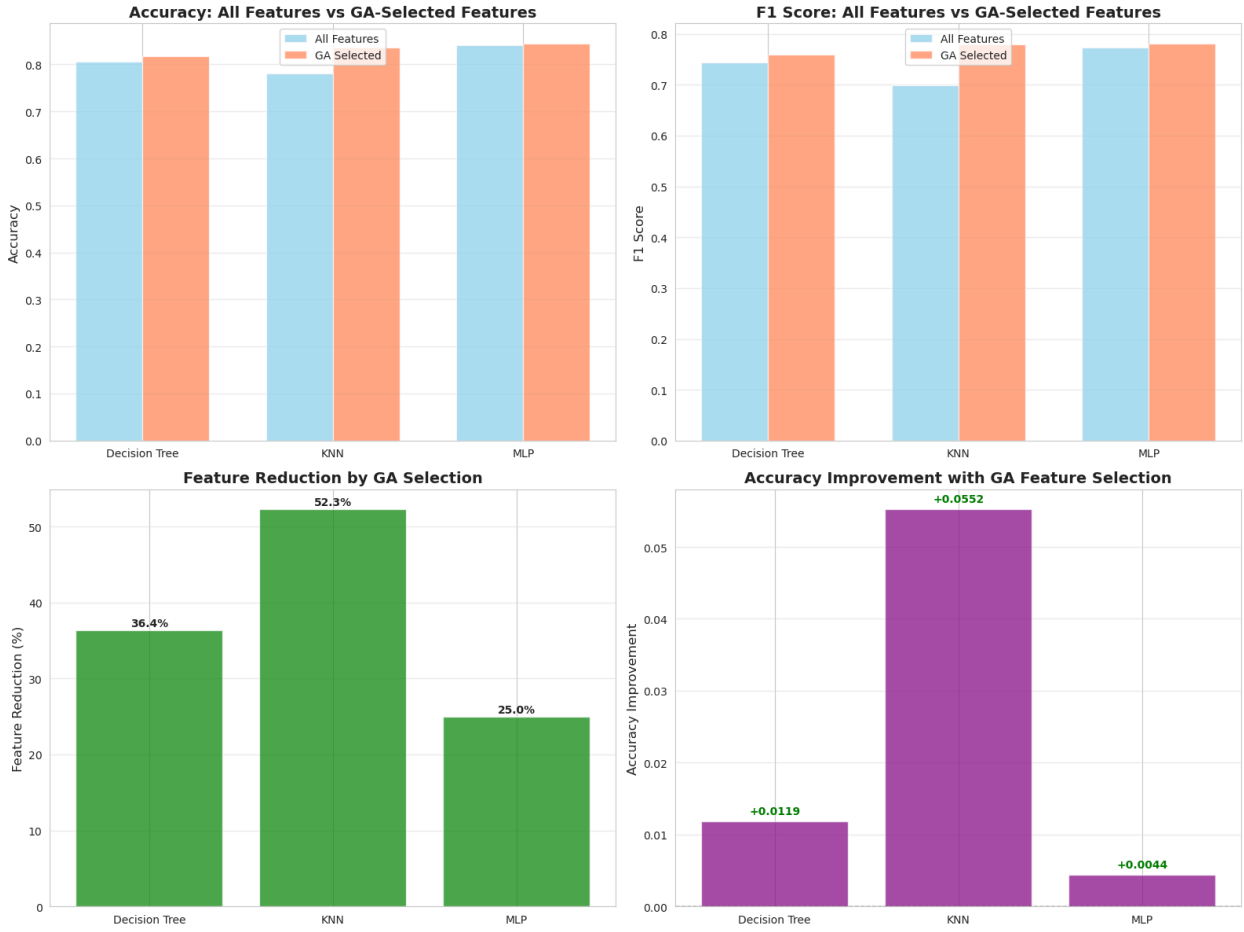
The classification report and confusion matrix show a similar pattern to before: the model is more precise and better at recalling class 0 than class 1. The key takeaway is that feature selection via GA achieved a slight performance gain while likely reducing the number of features used.

Confusion matrix



This confusion matrix shows how well the neural network's predictions match reality. The model correctly predicted the negative class (0) 13,550 times (top-left) and correctly predicted the positive class (1) 6,625 times (bottom-right)—those are the successes. However, it made two types of mistakes: it wrongly predicted negative 1,483 times (top-right, false positives) and missed 2,220 actual positives (bottom-left, false negatives). So, while it's better at catching negatives, it's still missing a meaningful number of actual positives

Impact of GA feature selection on performance



GA selection showed incremental yet promising improvements on feature selection.

In Conclusion,

KNN was selected first as our highest ranking model due to having really great accuracy whilst maintaining efficiency by using the fewest features, offering an optimal balance of simplicity and interpretability. MLP was second because it had the highest accuracy, however it was held back by its great computational cost and complexity.